



**Univerzitet u Nišu
Elektronski fakultet
Katedra za računarstvo**



Master rad

**Komparativna analiza REST i GraphQL
pristupa u razvoju Web API sistema u .NET
okruženju**

Mentor:

doc. dr Petar Rajković

Student:

Miloš Veljanovski

Niš, jun 2026.

Univerzitet u Nišu
Elektronski fakultet
Katedra za računarstvo

Komparativna analiza REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju

Comparative Analysis of REST and GraphQL Approaches in the Development of Web API Systems in the .NET Environment

Master rad
Studijski program: Računarstvo i informatika
Modul: Softversko inženjerstvo

Student: Miloš Veljanovski, br. ind. 1559

Mentor: doc. dr Petar Rajković

Zadatak: Cilj ovog rada jeste analiza i poređenje REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju, sa posebnim fokusom na njihove arhitektonske karakteristike, način razmene podataka i performanse u različitim scenarijima korišćenja. U okviru rada razvijena je aplikacija CommerceHub koja implementira oba pristupa nad istom poslovnom logikom, istim modelom domena i istom bazom podataka, čime se obezbeđuju jednaki uslovi za poređenje. Na osnovu sprovedenih eksperimenata i merenja izvršenih korišćenjem NBomber alata, analiziraju se metrike poput vremena obrade zahteva, latencije, veličine odgovora i propusnog opsega, sa ciljem identifikovanja prednosti, nedostataka i pogodnih scenarija primene svakog od posmatranih pristupa.

Datum prijave rada: 22.04.2026.

Datum predaje rada: _____

Datum odbrane rada: _____

Komisija za ocenu i odbranu:

1. Predsednik Komisije

2. Član

3. Član

Komparativna analiza REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju

SAŽETAK

Savremene aplikacije sve češće koriste Web API sloj kao osnovni mehanizam komunikacije između klijentskih aplikacija i serverske poslovne logike. U tom kontekstu, REST i GraphQL predstavljaju dva najzastupljenija pristupa u izgradnji API sistema, pri čemu svaki od njih polazi od različitih principa organizacije podataka i komunikacije. REST se zasniva na resursno orijentisanom modelu i unapred definisanim endpoint-ima, dok GraphQL uvodi strogo tipiziranu šemu i omogućava klijentu da precizno definiše strukturu podataka koje želi da dobije. Zbog toga izbor između ova dva pristupa nije samo pitanje tehnologije, već arhitektonska odluka koja može uticati na performanse, fleksibilnost i održivost sistema.

Cilj ovog rada jeste komparativna analiza REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju. U tu svrhu razvijena je aplikacija CommerceHub koja implementira oba pristupa nad istim poslovnim domenom, istim modelom podataka i istom bazom podataka, čime su obezbeđeni jednaki uslovi za poređenje. REST deo sistema realizovan je korišćenjem ASP.NET Core Minimal API tehnologije, dok je GraphQL implementiran upotrebom HotChocolate platforme. Kao podrška razvoju i posmatranju sistema korišćeni su .NET Aspire i OpenTelemetry mehanizmi, dok je za eksperimentalnu evaluaciju performansi primenjen NBomber alat za opterećenje i merenje.

Praktični deo rada obuhvata definisanje i izvođenje više reprezentativnih scenarija korišćenja, uključujući jednostavno čitanje podataka, rad sa složenijim objektima, problem prekomernog ili nedovoljnog preuzimanja podataka, rad sa listama entiteta i kombinovane operacije upisa i čitanja. Na osnovu prikupljenih rezultata analiziraju se metrike kao što su latencija, veličina odgovora i ponašanje sistema pod opterećenjem, sa ciljem identifikovanja prednosti i ograničenja oba pristupa. Dobijeni rezultati omogućavaju objektivnije sagledavanje razlika između REST i GraphQL tehnologija i pružaju smernice za njihov izbor u zavisnosti od zahteva konkretnog sistema.

Ključne reči: REST, GraphQL, Web API, ASP.NET Core, HotChocolate, performanse, NBomber, .NET Aspire, komparativna analiza

Comparative Analysis of REST and GraphQL Approaches in the Development of Web API Systems in the .NET Environment

ABSTRACT

Modern applications increasingly rely on the Web API layer as the primary mechanism for communication between client applications and server-side business logic. In this context, REST and GraphQL represent two of the most widely adopted approaches to API development, each based on different principles of data organization and communication. REST follows a resource-oriented model with predefined endpoints, whereas GraphQL introduces a strongly typed schema that allows clients to precisely specify the structure of the data they wish to retrieve. Consequently, the choice between these approaches is not merely a technological decision but also an architectural one that can significantly influence system performance, flexibility, and maintainability.

The objective of this thesis is to provide a comparative analysis of REST and GraphQL approaches in the development of Web API systems within the .NET environment. To achieve this goal, a software system named CommerceHub was developed, implementing both approaches over the same business domain, data model, and database, thereby ensuring identical conditions for comparison. The REST implementation is based on ASP.NET Core Minimal APIs, while GraphQL is implemented using the HotChocolate framework. As supporting technologies, .NET Aspire and OpenTelemetry were used for application orchestration and observability, while NBomber was employed for performance testing and experimental evaluation.

The practical part of the thesis includes the design and execution of several representative usage scenarios, including simple data retrieval, operations involving complex object graphs, over-fetching and under-fetching situations, paginated entity listings, and combined write-and-read workflows. Based on the collected results, metrics such as latency, response payload size, and system behavior under load are analyzed in order to identify the strengths and limitations of both approaches. The results obtained provide a more objective understanding of the differences between REST and GraphQL technologies and offer practical guidelines for selecting the most suitable approach depending on the requirements of a particular system.

Keywords: REST, GraphQL, Web API, ASP.NET Core, HotChocolate, performance, NBomber, .NET Aspire, comparative analysis

Sadržaj

1. Uvod.....	7
1.1. Predmet rada.....	7
1.2. Motivacija za izbor teme	8
1.3. Cilj rada	8
1.4. Metodologija rada.....	9
2. Teorijske osnove Web API sistema	10
2.1. Uloga Web API sloja u savremenim aplikacijama	10
2.2. HTTP, JSON i request-response model	12
2.3. REST pristup	13
2.3.1. Osnovni principi REST-a	14
2.3.2. REST u ASP.NET Core okruženju	14
2.3.3. Prednosti REST pristupa.....	15
2.3.4. Ograničenja REST pristupa	16
2.4. GraphQL pristup.....	17
2.4.1. Osnovni koncepti GraphQL-a	17
2.4.2. GraphQL kao klijentski definisan upitni model	18
2.4.3. Prednosti GraphQL pristupa	19
2.4.4. Ograničenja GraphQL pristupa.....	20
2.5. Komparativni pregled REST i GraphQL pristupa	21
3. Tehnološki okvir rada	23
3.1. .NET platforma i ASP.NET Core	23
3.2. ASP.NET Core Minimal APIs	24
3.3. HotChocolate GraphQL.....	25
3.4. Entity Framework Core i PostgreSQL	26
3.5. .NET Aspire i observability podrška	27
3.6. NBomber kao alat za benchmark testiranje.....	28
4. Projektovanje praktičnog sistema CommerceHub.....	29
4.1. Namena praktičnog sistema	29
4.2. Poslovni domen	30
4.3. Model baze podataka.....	31
4.4. Arhitektura rešenja	32
4.5. Zajednički poslovni sloj.....	34

4.6. Data seeding	35
4.7. Pokretanje sistema kroz .NET Aspire	36
5. Implementacija REST i GraphQL API slojeva	37
5.1. Implementacija REST API-ja	37
5.2. Primer REST operacije za čitanje proizvoda	38
5.3. Primer REST operacije za listu proizvoda	40
5.4. Implementacija GraphQL API-ja	41
5.5. Primer GraphQL upita za proizvod	42
5.6. Primer GraphQL upita sa minimalnim skupom polja	44
5.7. Usklađivanje REST i GraphQL odgovora	45
5.8. Razlike u implementacionom modelu	46
6. Metodologija benchmark testiranja	48
6.1. Cilj benchmark testiranja	48
6.2. Testno okruženje	49
6.3. Pravila fer poređenja	50
6.4. Metrike	51
6.5. Profil opterećenja	52
6.6. Benchmark scenariji	53
6.7. Ograničenja metodologije	54
7. Rezultati i analiza benchmark testiranja	55
7.1. Pregled rezultata	56
7.2. Scenario 1: Simple GET	57
7.3. Scenario 2: Deep graph fetch	58
7.4. Scenario 3: Over-fetching vs minimal fetching	59
7.5. Scenario 4: Paged product list	60
7.6. Scenario 5: Write + read-back	62
7.7. Analiza rezultata i poređenje sa teorijskim očekivanjima	63
8. Diskusija i zaključak	65
8.1. Diskusija rezultata	65
8.2. Praktične smernice za izbor REST ili GraphQL pristupa	66
8.3. Zaključak	67
9. Literatura	68
10. Listing	69

1. Uvod

1.1. Predmet rada

Savremeni razvoj softverskih sistema u velikoj meri se oslanja na Web API sloj kao osnovni mehanizam komunikacije između klijentskih aplikacija i serverske poslovne logike. Bez obzira na to da li je reč o web aplikacijama, mobilnim aplikacijama, administrativnim panelima, integracijama sa partnerskim sistemima ili internim servisima, API sloj predstavlja ugovor preko koga se funkcionalnosti sistema izlažu spoljašnjim korisnicima ili drugim softverskim komponentama. Zbog toga način na koji je API projektovan direktno utiče na upotrebljivost, performanse, održivost i evoluciju celokupnog sistema.

U praksi se kao jedan od najzastupljenijih pristupa razvoju Web API-ja već duže vreme koristi REST. REST pristup zasniva se na resursima, HTTP metodama i unapred definisanim endpoint-ima koji klijentima vraćaju određene reprezentacije podataka. Takav model je jednostavan za razumevanje, dobro se uklapa u postojeću HTTP infrastrukturu i prirodno podržava alate za testiranje, keširanje, dokumentovanje i nadzor API-ja. Zbog toga je REST postao dominantan obrazac za razvoj javnih i internih Web API sistema.

Sa druge strane, poslednjih godina sve značajnije mesto zauzima GraphQL, pristup koji polazi od drugačije ideje: umesto većeg broja endpoint-a sa unapred definisanim oblikom odgovora, server izlaže strogo tipiziranu šemu, a klijent u samom zahtevu navodi koja polja i koje povezane podatke želi da dobije. Takav model omogućava veću fleksibilnost na strani klijenta i može smanjiti problem prekomernog preuzimanja podataka, ali istovremeno uvodi dodatnu složenost u pogledu izvršavanja upita, projektovanja šeme, autorizacije, keširanja i optimizacije performansi.

Predmet ovog rada jeste komparativna analiza REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju. Rad ne posmatra ove pristupe samo na teorijskom nivou, već ih upoređuje kroz praktičnu implementaciju iste aplikacije, nad istim poslovnim domenom, istom bazom podataka i istom poslovnom logikom. Na taj način se izbegava česta slabost površnih poređenja, gde se REST i GraphQL upoređuju na osnovu različitih primera, različitih implementacija ili različitog obima podataka. Umesto toga, cilj je da se oba pristupa dovedu u što sličnije uslove i da se zatim analiziraju njihove razlike kroz konkretne i merljive rezultate.

Poseban značaj rada ogleda se u tome što je poređenje realizovano u .NET ekosistemu, korišćenjem savremenih tehnologija kao što su ASP.NET Core Minimal APIs za REST implementaciju, HotChocolate za GraphQL implementaciju, Entity Framework Core za rad sa bazom podataka, PostgreSQL kao relaciona baza, .NET Aspire kao podrška za lokalnu orkestraciju i NBomber kao alat za benchmark testiranje. Time rad povezuje teorijsku analizu API paradigmi sa realnim tehnološkim okruženjem koje se koristi u savremenom backend razvoju.

1.2. Motivacija za izbor teme

Izbor između REST i GraphQL pristupa često se u praksi posmatra kroz pojednostavljene tvrdnje. REST se neretko opisuje kao jednostavniji, brži i stabilniji pristup, dok se GraphQL predstavlja kao fleksibilnije i modernije rešenje koje omogućava klijentu da dobije tačno one podatke koji su mu potrebni. Iako obe tvrdnje imaju određeno utemeljenje, one nisu dovoljne za donošenje ozbiljne arhitektonske odluke. U realnim sistemima performanse i upotrebljivost API-ja ne zavise samo od izabrane paradigme, već i od načina implementacije, oblika podataka, složenosti domena, potreba klijentskih aplikacija, količine prenetih podataka, upotrebljenih biblioteka i samog runtime okruženja.

Motivacija za ovaj rad proističe upravo iz potrebe da se REST i GraphQL ne porede samo opisno, već i eksperimentalno. U mnogim člancima i diskusijama mogu se pronaći opšti zaključci o tome kada je jedan pristup bolji od drugog, ali su takva poređenja često neujednačena. Na primer, REST implementacija može biti realizovana kroz namenski optimizovan endpoint, dok GraphQL implementacija može biti napisana naivno i bez odgovarajuće projekcije podataka. U drugom slučaju GraphQL može koristiti selektivne upite, dok REST vraća prevelike DTO objekte koji nisu prilagođeni konkretnom scenariju. Takva poređenja ne daju dovoljno pouzdanu osnovu za zaključivanje, jer nije jasno da li izmerene razlike potiču od same API paradigme ili od različitog kvaliteta implementacije.

Upravo zato je u ovom radu razvijena aplikacija CommerceHub, koja omogućava da oba pristupa budu implementirana nad istim modelom podataka i istom poslovnom logikom. Domen aplikacije izabran je tako da bude dovoljno jednostavan za razumevanje, ali dovoljno bogat da omogući različite scenarije poređenja. Sistem obuhvata proizvode, kategorije, kupce, porudžbine, stavke porudžbine, zalihe i dobavljače. Takav domen prirodno sadrži i jednostavne operacije čitanja, i liste entiteta, i povezane objekte, i ugnježdene podatke, što ga čini pogodnim za analizu razlika između REST i GraphQL pristupa.

Dodatna motivacija za izbor teme jeste praktična relevantnost ovakve analize za .NET razvoj. ASP.NET Core danas omogućava vrlo efikasnu implementaciju REST API-ja kroz Minimal APIs pristup, dok HotChocolate predstavlja jedno od najzrelijih GraphQL rešenja u .NET ekosistemu. To znači da oba pristupa mogu biti realizovana na moderan i relevantan način, bez oslanjanja na zastarele obrasce ili neprirodne implementacije. Poređenjem ova dva pristupa u istom .NET okruženju dobija se jasnija slika o tome kako se teorijske razlike između REST-a i GraphQL-a odražavaju na konkretan kod, performanse i oblik odgovora.

1.3. Cilj rada

Cilj ovog rada jeste da se izvrši komparativna analiza REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju, uz oslanjanje na praktičnu implementaciju i merljive rezultate. Rad ne nastoji da unapred proglasi jedan pristup univerzalno boljim,

već da pokaže u kojim scenarijima svaki od njih dolazi do izražaja, koje prednosti pruža i kakva ograničenja uvodi.

Da bi se taj cilj ostvario, najpre je potrebno teorijski objasniti osnovne principe Web API razvoja, REST pristupa i GraphQL pristupa. Teorijski deo rada daje osnovu za razumevanje ključnih razlika između ova dva modela komunikacije. REST polazi od resursa i standardnih HTTP operacija, dok GraphQL polazi od tipizirane šeme i klijentski definisanih upita. Ta razlika nije samo sintaksna, već utiče na način modelovanja API-ja, oblik odgovora, odgovornost servera i fleksibilnost klijenta.

Nakon teorijskog dela, cilj je da se prikaže praktična implementacija sistema CommerceHub. U okviru tog sistema implementirana su dva API sloja: REST API zasnovan na ASP.NET Core Minimal APIs tehnologiji i GraphQL API zasnovan na HotChocolate biblioteci. Oba API sloja oslanjaju se na isti domen, isti application sloj i istu PostgreSQL bazu podataka. Time se postiže da razlike u ponašanju sistema ne budu posledica različite poslovne logike ili različitog modela podataka, već prvenstveno različitog načina izlaganja i obrade API zahteva.

Treći cilj rada jeste eksperimentalna evaluacija ova dva pristupa kroz više benchmark scenarija. Korišćenjem NBomber alata meri se ponašanje REST i GraphQL implementacije u scenarijima koji obuhvataju jednostavno čitanje pojedinačnog entiteta, dohvaćanje složenijeg objektnog grafa, poređenje prekomernog i selektivnog preuzimanja podataka, rad sa paginiranom listom proizvoda i kombinovanu operaciju upisa i naknadnog čitanja. Za svaki scenario analiziraju se latencija, percentili, veličina odgovora i stabilnost izvršavanja pod opterećenjem.

Konačni cilj rada jeste formulisanje praktičnih zaključaka o primeni REST i GraphQL pristupa. Umesto apstraktne tvrdnje da je jedan pristup bolji, rad nastoji da pokaže da izbor zavisi od konkretnog tipa zahteva. REST se očekivano pokazuje kao pogodan za stabilne i unapred definisane oblike odgovora, dok GraphQL ima naročitu vrednost u scenarijima gde klijent zaista koristi mogućnost da traži samo podskup potrebnih polja. Takav zaključak može biti koristan ne samo u akademskom smislu, već i u praktičnom projektovanju budućih Web API sistema.

1.4. Metodologija rada

Metodologija rada zasniva se na kombinaciji teorijske analize, praktične implementacije i eksperimentalnog merenja. Prvi deo metodologije obuhvata analizu osnovnih pojmova i principa REST i GraphQL pristupa. U tom delu razmatraju se razlike u načinu organizacije API sloja, strukturi zahteva, strukturi odgovora, fleksibilnosti klijenta, složenosti servera i tipičnim scenarijima primene. Posebno se obraća pažnja na probleme kao što su over-fetching, under-fetching, N+1 problem, keširanje, projektovanje šeme i stabilnost API ugovora.

Drugi deo metodologije predstavlja razvoj praktičnog sistema CommerceHub. Sistem je projektovan kao aplikacija za upravljanje proizvodima i porudžbinama, sa dovoljno bogatim modelom podataka da omogući smislene scenarije poređenja. U implementaciji su jasno razdvojeni domen, application sloj, infrastructure sloj i dva Web

API sloja. REST i GraphQL API ne sadrže sopstvenu poslovnu logiku, već koriste zajedničke upite, komande, entitete i bazu podataka. Takva organizacija omogućava metodološki korektnije poređenje, jer se oba pristupa oslanjaju na istu osnovu.

Treći deo metodologije odnosi se na definisanje benchmark scenarija. Scenariji su izabrani tako da pokriju tipične situacije u kojima se REST i GraphQL razlikuju. Jednostavno čitanje pojedinačnog proizvoda koristi se kao osnovni test per-request overhead-a. Dohvatanje složenijeg objektnog grafa koristi se za analizu rada sa ugnježenim podacima. Scenario prekomernog i minimalnog preuzimanja podataka ispituje jednu od glavnih teorijskih prednosti GraphQL-a. Paginirana lista proizvoda koristi se za poređenje ponašanja pri radu sa većim brojem entiteta. Konačno, write-read scenario meri ponašanje sistema kada se kombinuju operacija upisa i naknadno čitanje kreiranog objekta.

Benchmark testiranje izvršeno je korišćenjem NBomber alata. Svaki scenario se izvršava pod definisanim opterećenjem, uz merenje broja uspešnih zahteva, latencije, percentila, maksimalnog vremena odgovora i veličine prenetih podataka. Pre merenja se koristi warm-up faza kako bi se smanjio uticaj hladnog starta, JIT kompilacije, inicijalizacije Entity Framework Core modela, uspostavljanja konekcija i inicijalnog zagrevanja GraphQL izvršnog okruženja. REST i GraphQL testovi izvršavaju se u odvojenim prolazima kako bi se izbegla direktna konkurencija za bazu podataka i load generator tokom istog testa.

Metodologija uključuje i jasno navođenje ograničenja. Testiranje se izvršava u lokalnom okruženju, bez realne mrežne latencije, autentifikacije, autorizacije, keširanja, kompresije odgovora i TLS sloja. Dataset je dovoljno veliki da bude relevantniji od trivijalnog primera, ali ne predstavlja produkcionni obim podataka. Zbog toga rezultati ne predstavljaju univerzalnu istinu o REST-u i GraphQL-u, već merljive nalaze za konkretnu implementaciju, konkretan domen i konkretno testno okruženje.

2. Teorijske osnove Web API sistema

2.1. Uloga Web API sloja u savremenim aplikacijama

Web API sloj predstavlja jedan od najvažnijih delova savremenih softverskih sistema. Njegova osnovna uloga je da omogući komunikaciju između klijentskih aplikacija i serverske poslovne logike na standardizovan, predvidiv i kontrolisan način. Klijentske aplikacije danas mogu biti veoma različite: web frontend aplikacije, mobilne aplikacije, desktop aplikacije, administrativni portali, partnerski sistemi ili drugi backend servisi. Bez obzira na njihovu prirodu, svima im je potreban način da pristupe funkcionalnostima sistema, pošalju podatke, dobiju rezultate i reaguju na promene u poslovnom domenu.

U tradicionalnim aplikacijama korisnički interfejs i poslovna logika često su bili čvrsto povezani u okviru jedne aplikacije. Međutim, razvojem web-a, mobilnih platformi i distribuiranih sistema, ova veza se sve više razdvaja. Backend sistemi postaju nezavisni pružaoci podataka i funkcionalnosti, dok klijentske aplikacije postaju potrošači tih

funkcionalnosti. API sloj je upravo granica između ta dva sveta. On određuje šta sistem nudi, na koji način se tim funkcionalnostima pristupa i u kom obliku se podaci razmenjuju.

Zbog toga API sloj nije samo tehnički dodatak aplikaciji. On predstavlja javni ili interni ugovor sistema. Dobro projektovan API olakšava razvoj klijentskih aplikacija, smanjuje mogućnost nesporazuma između timova, pojednostavljuje testiranje i omogućava lakšu evoluciju sistema. Loše projektovan API, sa druge strane, može postati dugoročno ograničenje. Ako endpoint-i nisu dosledni, ako odgovori sadrže previše ili premalo podataka, ako se promene uvode bez jasne strategije verzionisanja ili ako performanse nisu zadovoljavajuće, API sloj može značajno usporiti razvoj i održavanje celog sistema.

Jedan od ključnih izazova u projektovanju API sloja jeste pronalaženje ravnoteže između stabilnosti i fleksibilnosti. Stabilan API omogućava klijentima da se oslone na predvidiv ugovor i da ne moraju stalno da prilagođavaju svoj kod promenama na serveru. Sa druge strane, poslovni zahtevi i korisnički interfejsi se menjaju. Klijenti često vremenom zahtevaju nove kombinacije podataka, nove filtracije, nove projekcije i drugačije oblike odgovora. Ukoliko je API previše krut, svaka promena na klijentu može zahtevati novi endpoint ili izmenu postojećeg odgovora. Ukoliko je previše fleksibilan, server može postati složeniji za optimizaciju, kontrolu i zaštitu.

REST i GraphQL predstavljaju dva različita odgovora na ovaj problem. REST nudi stabilan i resursno orijentisan model u kome server unapred definiše endpoint-e i oblik odgovora. GraphQL nudi fleksibilniji model u kome server definiše šemu, ali klijent bira konkretna polja koja želi da dobije. Ova razlika direktno utiče na API dizajn, performanse, količinu prenetih podataka i iskustvo razvoja klijentskih aplikacija. Zbog toga je razumevanje uloge API sloja neophodno pre detaljnijeg poređenja REST i GraphQL pristupa.

Web API sloj takođe ima važnu ulogu u očuvanju granica između različitih delova sistema. Iako klijentska aplikacija može imati potrebu za složenim prikazom podataka, API ne treba nužno da direktno izlaže unutrašnji model baze ili domenske entitete. U dobro projektovanom sistemu API sloj najčešće vraća posebne response modele ili projekcije prilagođene potrebama klijenta. Time se sprečava da unutrašnja struktura aplikacije procuri ka spoljnim korisnicima i oteža kasnije promene. Ova ideja je jednako važna i kod REST-a i kod GraphQL-a, iako se realizuje na različite načine.

Kod REST pristupa server najčešće definiše DTO modele koji predstavljaju oblik odgovora za određeni endpoint. Na primer, endpoint za listu proizvoda može vratiti kraći prikaz proizvoda, dok endpoint za detalje proizvoda može vratiti širi skup podataka. Kod GraphQL-a se slična fleksibilnost postiže kroz selection set, jer klijent može izabrati koja polja želi iz šeme. Ipak, i u GraphQL-u je potrebno pažljivo odlučiti šta šema izlaže i kako su polja povezana sa stvarnom poslovnom logikom i bazom podataka. U oba slučaja API sloj predstavlja oblikovanu granicu sistema, a ne puki automatski prozor u bazu.

U kontekstu ovog rada, Web API sloj ima dvostruku ulogu. Sa jedne strane, on je teorijski predmet analize, jer se kroz njega posmatraju razlike između REST i GraphQL pristupa. Sa druge strane, on je i praktični eksperimentalni objekat, jer su oba pristupa implementirana u okviru iste aplikacije i zatim merena kroz benchmark scenarije. Takav pristup omogućava da se API sloj ne posmatra samo kroz principe i preporuke, već i kroz konkretno ponašanje sistema pod opterećenjem.

2.2. HTTP, JSON i request-response model

Većina savremenih Web API sistema zasniva se na HTTP protokolu i request-response modelu komunikacije. U tom modelu klijent šalje zahtev serveru, a server nakon obrade vraća odgovor. Zahtev obično sadrži HTTP metodu, URL, opcione query parametre, zaglavlja i telo zahteva. Odgovor sadrži status kod, zaglavlja i telo odgovora. Iako je ovaj model na prvi pogled jednostavan, upravo na njemu počiva najveći deo komunikacije u savremenim web i backend sistemima.

HTTP metode imaju važnu semantičku ulogu. Najčešće korišćene metode su GET, POST, PUT, PATCH i DELETE. GET se obično koristi za čitanje podataka, POST za kreiranje novih resursa ili izvršavanje poslovnih operacija, PUT za potpunu zamenu resursa, PATCH za delimičnu izmenu, a DELETE za brisanje. REST pristup se snažno oslanja na ovu semantiku i nastoji da API operacije izrazi kroz kombinaciju resursa i odgovarajućih HTTP metoda. Na primer, dohvatanje proizvoda može biti predstavljeno kao GET zahtev na rutu `/products/{id}`, dok kreiranje porudžbine može biti POST zahtev na rutu `/orders`.

Status kodovi predstavljaju drugi važan deo HTTP modela. Oni omogućavaju serveru da klijentu jasno saopšti rezultat obrade zahteva. Status kod 200 obično označava uspešnu obradu, 201 kreiranje resursa, 400 neispravan zahtev, 401 neautentifikovan pristup, 403 zabranjen pristup, 404 nepostojeći resurs, a 500 neočekivanu serversku grešku. Dobro korišćenje status kodova doprinosi jasnijem API ugovoru i olakšava klijentskoj aplikaciji da pravilno reaguje na različite ishode.

JSON format se u praksi najčešće koristi za telo zahteva i odgovora. Razlog za to je njegova čitljivost, jednostavnost i dobra podrška u gotovo svim programskim jezicima i alatima. JSON prirodno predstavlja objekte, nizove, brojeve, tekstualne vrednosti, logičke vrednosti i null vrednosti, što ga čini pogodnim za razmenu strukturiranih podataka između klijenta i servera. I REST i GraphQL najčešće koriste JSON kao transportni format, iako se razlikuju u načinu na koji se struktura odgovora definiše.

Kod REST pristupa, strukturu JSON odgovora određuje server. Klijent poziva određeni endpoint i dobija unapred definisan oblik odgovora. Na primer, endpoint za proizvod može vratiti identifikator, naziv, SKU, opis, cenu i kategoriju. Ako klijentu treba samo naziv i cena, on će ipak dobiti ceo odgovor, osim ako server ne obezbedi poseban endpoint ili neki mehanizam za projekciju polja. Ovo je osnova problema over-fetching-a, odnosno preuzimanja više podataka nego što je klijentu potrebno.

Kod GraphQL pristupa, HTTP se takođe koristi kao transportni protokol, ali se većina semantike zahteva nalazi u telu GraphQL upita. Klijent šalje upit na najčešće jedan endpoint, na primer `/graphql`, i u telu zahteva navodi koja polja želi. Server zatim izvršava upit u odnosu na GraphQL šemu i vraća JSON odgovor koji odgovara strukturi traženog selection set-a. Time se oblik odgovora pomera iz unapred definisanog endpoint-a u sam klijentski zahtev.

Ova razlika ima značajne posledice. REST API je često pregledniji kroz same URL rute, jer se iz kombinacije metode i putanje može zaključiti koja operacija se izvršava. GraphQL API je fleksibilniji, ali se stvarna semantika zahteva nalazi u query tekstu. To

znači da se analiza, keširanje, autorizacija i observability kod GraphQL-a moraju posmatrati drugačije nego kod REST-a. Na primer, dva GraphQL zahteva mogu ići na isti HTTP endpoint, ali tražiti potpuno različite podatke i imati različitu cenu izvršavanja.

Request-response model je važan i za merenje performansi. Latencija jednog API zahteva obuhvata vreme od slanja zahteva do primanja odgovora. U to vreme ulaze mrežni troškovi, obrada na serveru, pristup bazi, serijalizacija odgovora i drugi runtime troškovi. U lokalnom benchmark okruženju mrežni troškovi su minimalni, pa se veći deo razlike između REST i GraphQL pristupa može pripisati serverskoj obradi, obliku upita, pristupu podacima i serijalizaciji.

U ovom radu HTTP i JSON predstavljaju zajedničku osnovu za oba pristupa. I REST i GraphQL komuniciraju preko HTTP-a i vraćaju JSON odgovore, ali se razlikuju u organizaciji zahteva, oblikovanju odgovora i stepenu fleksibilnosti koji se daje klijentu. Upravo zato se njihovo poređenje ne svodi na poređenje različitih protokola, već na poređenje dva različita modela korišćenja istog osnovnog web komunikacionog mehanizma.

2.3. REST pristup

REST, odnosno Representational State Transfer, predstavlja arhitektonski stil za projektovanje sistema zasnovanih na razmeni reprezentacija resursa preko mreže. Iako se u praksi često poistovećuje sa bilo kojim HTTP API-jem koji koristi JSON, REST je širi koncept koji podrazumeva određeni skup principa i ograničenja. Osnovna ideja REST pristupa jeste da se sistem posmatra kroz resurse, pri čemu svaki resurs ima svoj identifikator, najčešće u obliku URI-ja, a klijent nad tim resursima izvršava operacije korišćenjem standardnih HTTP metoda.

U kontekstu Web API razvoja, REST je postao jedan od najrasprostranjenijih pristupa zato što se prirodno nadovezuje na postojeći model rada web-a. HTTP već poseduje standardne metode, status kodove, zaglavlja, mehanizme za keširanje i široku podršku u alatima, bibliotekama, serverima i klijentskim platformama. REST koristi upravo te mehanizme i zbog toga ne zahteva poseban transportni protokol ili potpuno novu infrastrukturu. API projektovan u REST stilu obično ima više endpoint-a, pri čemu svaki endpoint predstavlja određeni resurs ili kolekciju resursa.

Na primer, u sistemu za naručivanje proizvoda endpoint *GET /products/{id}* može predstavljati zahtev za dohvaćanje jednog proizvoda, *GET /products* zahtev za dohvaćanje liste proizvoda, *POST /orders* kreiranje nove porudžbine, dok *GET /orders/{id}* predstavlja dohvaćanje detalja konkretne porudžbine. Klijent iz same kombinacije HTTP metode i URL putanje može relativno lako da razume šta se od sistema traži. Ova eksplicitnost predstavlja jednu od najvećih praktičnih prednosti REST pristupa.

2.3.1. Osnovni principi REST-a

Jedan od osnovnih principa REST-a jeste resursna orijentisanost. Umesto da API bude organizovan oko udaljenih procedura ili funkcija, on se organizuje oko resursa koji postoje u domenu sistema. Resurs može predstavljati proizvod, korisnika, porudžbinu, kategoriju, komentar, dokument ili bilo koji drugi pojam koji ima poslovni ili informativni značaj. Svaki resurs se identifikuje putem URI-ja, dok se operacije nad njim izražavaju korišćenjem HTTP metoda.

Drugi važan princip jeste razdvajanje reprezentacije resursa od njegovog unutrašnjeg modela. Kada klijent zatraži neki resurs, server ne mora da mu vrati direktan prikaz entiteta iz baze podataka. Umesto toga, vraća reprezentaciju resursa prilagođenu API ugovoru. U praksi je ta reprezentacija najčešće JSON objekat, ali teorijski može biti i XML, HTML ili neki drugi format. Ovo razdvajanje omogućava serveru da menja unutrašnju implementaciju bez narušavanja spoljašnjeg API ugovora, pod uslovom da format odgovora ostane stabilan.

Treći princip odnosi se na korišćenje standardne HTTP semantike. REST API bi trebalo da koristi HTTP metode na način koji odgovara njihovom značenju. GET zahtevi treba da budu bezbedni i da ne menjaju stanje sistema. POST se koristi za kreiranje resursa ili izvršavanje poslovnih operacija koje menjaju stanje. PUT i PATCH se koriste za izmene, dok DELETE označava brisanje. Ispravno korišćenje HTTP status kodova takođe je važno, jer klijentu omogućava da jasno razlikuje uspešne, neispravne, neautorizovane ili neuspele zahteve.

Jedan od često pominjanih REST principa jeste stateless komunikacija. To znači da svaki zahtev treba da sadrži sve informacije koje su serveru potrebne da ga obradi. Server ne bi trebalo da zavisi od stanja prethodnih zahteva u okviru sesije. Ovakav pristup pojednostavljuje skaliranje, jer bilo koja instanca servera može obraditi bilo koji zahtev, pod uslovom da ima pristup potrebnim podacima i konfiguraciji. U praksi se stanje korisnika najčešće prenosi kroz tokene, zaglavlja ili druge eksplicitne mehanizme, a ne kroz implicitnu serversku sesiju.

REST se takođe dobro uklapa sa keširanjem. Pošto se čitanje podataka najčešće vrši putem GET zahteva, HTTP infrastruktura može koristiti standardne mehanizme kao što su cache-control zaglavlja, ETag vrednosti i reverse proxy keširanje. Ovo je naročito važno za javne API-je, statičnije podatke ili scenarije sa velikim brojem ponovljenih čitanja. Iako keširanje nije automatski rešeno samim izborom REST-a, REST se prirodno oslanja na postojeće HTTP mogućnosti, što može biti značajna prednost u odnosu na pristupe gde se sva semantika zahteva nalazi unutar tela POST zahteva.

2.3.2. REST u ASP.NET Core okruženju

ASP.NET Core pruža veoma razvijenu podršku za izgradnju REST API sistema. Tradicionalno, REST API-ji u .NET okruženju često su implementirani korišćenjem kontrolera, atributa za rute, action metoda i model binding mehanizama. Međutim, novije verzije ASP.NET Core platforme uvode i Minimal APIs pristup, koji omogućava definisanje

HTTP endpoint-a uz manje ceremonijalnog koda. Upravo ovaj pristup korišćen je u praktičnom delu rada za REST implementaciju sistema CommerceHub.

Minimal APIs omogućavaju da se endpoint-i definišu direktno kroz mapiranje ruta i handler funkcija. Na primer, MapGet, MapPost, MapPut i slične metode koriste se za povezivanje HTTP metoda i ruta sa odgovarajućim funkcijama. Ovakav stil razvoja pogodan je za API slojeve koji treba da budu tanki i eksplicitni. U praktičnoj implementaciji ovog rada REST endpoint-i ne sadrže složenu poslovnu logiku, već prihvataju parametre iz zahteva, kreiraju odgovarajuće query ili command objekte i prosleđuju ih application sloju.

Ovakav pristup ima nekoliko prednosti. Prvo, endpoint-i ostaju kratki i pregledni. Drugo, poslovna logika nije vezana za HTTP sloj, već se nalazi u application handler-ima. Treće, isti application sloj može biti iskorišćen i od strane GraphQL API-ja, što je veoma važno za fer poređenje. Kada bi REST i GraphQL implementacije imale različitu poslovnu logiku, različite query modele ili različite načine pristupa bazi, rezultati benchmark testiranja bili bi teže interpretabilni. Zbog toga je razdvajanje API sloja od poslovne logike jedna od važnih projektantskih odluka u radu.

U REST implementaciji često se koriste DTO modeli, odnosno posebni modeli odgovora. Oni predstavljaju oblik podataka koji se vraća klijentu i ne moraju biti identični domenskim entitetima. Na primer, proizvod u bazi može imati relaciju ka kategoriji, dok REST odgovor može sadržati samo CategoryName kao tekstualno polje. Time se klijentu daje upravo onaj oblik podataka koji je predviđen API ugovorom. Ovaj pristup je jednostavan i efikasan, ali zahteva da server unapred definiše koji shape odgovora se vraća za svaki endpoint.

U kontekstu poređenja sa GraphQL-om, ova osobina REST-a je veoma značajna. REST endpoint može biti vrlo efikasan ako je response model dobro prilagođen konkretnom scenariju. Na primer, namenski endpoint za detalje porudžbine može jednim optimizovanim upitom vratiti porudžbinu, kupca i stavke porudžbine. Međutim, ako različiti klijenti traže različite kombinacije podataka, broj endpoint-a i DTO modela može rasti. Tada REST može postati manje fleksibilan, jer svaka nova klijentska potreba potencijalno zahteva novi endpoint ili izmenu postojećeg odgovora.

2.3.3. Prednosti REST pristupa

Najveća prednost REST pristupa jeste jednostavnost. Većina programera, alata i platformi dobro razume HTTP metode, rute i status kodove. REST API je lako testirati kroz alate kao što su Postman, Bruno, curl ili Swagger UI. Iz same rute i HTTP metode često se može zaključiti šta endpoint radi. Ova transparentnost je veoma važna u timskom razvoju, dokumentovanju API-ja i integraciji sa spoljnim sistemima.

Druga važna prednost jeste zrelost ekosistema. REST je dugo prisutan u industriji i oko njega postoji veliki broj standardizovanih praksi. OpenAPI specifikacija omogućava formalno opisivanje REST API-ja, generisanje dokumentacije, klijentskih SDK biblioteka i testova. Reverse proxy serveri, API gateway-i, monitoring alati i sigurnosni mehanizmi često su već prilagođeni REST i HTTP modelu. To znači da izbor REST-a retko uvodi dodatnu infrastrukturnu složenost.

Treća prednost jeste efikasnost u scenarijima sa unapred poznatim oblikom odgovora. Ako server tačno zna koji podaci su potrebni za određenu operaciju, može pripremiti optimizovan DTO i odgovarajući SQL upit. U takvom slučaju REST endpoint može biti veoma brz, jer nema dodatni sloj parsiranja upita, validacije šeme ili dinamičkog izvršavanja selection set-a. Upravo ova osobina se pokazuje važnom u praktičnom delu rada, gde REST u više fiksnih scenarija ostvaruje nižu latenciju od GraphQL implementacije.

REST je takođe prirodan izbor za API-je kod kojih su operacije poslovno jasne i stabilne. Ako sistem ima ograničen broj klijenata, dobro poznate ekrane i predvidive potrebe za podacima, REST može pružiti odličan odnos jednostavnosti i performansi. Na primer, administrativni panel koji uvek prikazuje isti skup polja za proizvode, porudžbine i korisnike ne mora nužno imati potrebu za fleksibilnošću GraphQL-a. U takvom okruženju eksplicitni REST endpoint-i mogu biti lakši za održavanje.

Još jedna prednost REST-a jeste jednostavniji mentalni model sigurnosti i autorizacije. Pošto su operacije često razdvojene po endpoint-ima, lakše je primeniti pravila pristupa na nivou rute ili HTTP metode. Na primer, samo administratori mogu pozivati *DELETE /products/{id}*, dok obični korisnici mogu pozivati *GET /products*. Iako se i GraphQL može bezbedno implementirati, autorizacija je kod njega često složenija, jer jedan endpoint može sadržati veliki broj različitih polja, tipova i kombinacija upita.

2.3.4. Ograničenja REST pristupa

I pored svojih prednosti, REST pristup ima značajna ograničenja. Jedno od najpoznatijih je over-fetching, odnosno vraćanje više podataka nego što je klijentu potrebno. Pošto server unapred definiše oblik odgovora, klijent često nema mogućnost da traži samo određena polja. Ako endpoint za proizvod vraća identifikator, naziv, SKU, opis, cenu, kategoriju i druge informacije, klijent koji želi samo naziv i cenu ipak dobija ceo objekat. Kod malih odgovora ovo možda nije problem, ali kod većih objekata, sporijih mreža ili velikog broja zahteva može imati merljiv uticaj.

Drugi problem je under-fetching, odnosno situacija u kojoj jedan endpoint ne vraća dovoljno podataka, pa klijent mora da pošalje više zahteva da bi sastavio potreban prikaz. Na primer, klijent može najpre dohvatiti porudžbinu, zatim posebno kupca, zatim posebno proizvode iz stavki porudžbine. Više mrežnih poziva povećava latenciju i složenost klijentskog koda. REST ovaj problem često rešava uvođenjem namenski agregiranih endpoint-a, kao što je *GET /orders/{id}/details*, ali to povećava broj endpoint-a i zahteva dodatno održavanje.

Treće ograničenje odnosi se na evoluciju API-ja. Kada se potrebe klijenata promene, postojeći REST endpoint možda više ne odgovara idealno. Dodavanje novih polja može povećati payload za sve klijente, čak i one kojima nova polja nisu potrebna. Uklanjanje polja može narušiti kompatibilnost. Kreiranje novih endpoint-a može dovesti do dupliranja logike i većeg broja response modela. Zbog toga REST API-ji često zahtevaju pažljivo verzionisanje i dobru strategiju evolucije ugovora.

REST takođe može biti manje pogodan kada različiti klijenti imaju veoma različite potrebe. Mobilna aplikacija može želiti mali response zbog ograničene mreže, web

aplikacija može želeći širi response za bogat prikaz, dok partnerska integracija može koristiti samo nekoliko polja. Ako svi koriste isti REST endpoint, neki klijenti dobijaju previše podataka, a ako se za svakog klijenta prave posebni endpoint-i, API sloj postaje složeniji. Ovo je jedan od razloga zbog kojih je GraphQL postao popularan u sistemima sa više različitih frontend klijenata.

Na kraju, REST ne propisuje uvek jednoznačno kako treba modelovati složene poslovne operacije koje nisu jednostavno CRUD operacije nad resursima. U praksi se često pojavljuju endpoint-i koji više liče na akcije nego na resurse, na primer *POST /orders/{id}/cancel*, *POST /payments/confirm* ili *POST /reservations/commit*. Takvi endpoint-i mogu biti potpuno opravdani, ali pokazuju da realni REST API-ji često odstupaju od idealizovanog resursnog modela kako bi bolje izrazili poslovnu semantiku.

2.4. GraphQL pristup

GraphQL predstavlja pristup razvoju API-ja koji se zasniva na strogo tipiziranoj šemi i klijentski definisanim upitima. Za razliku od REST-a, gde server izlaže više endpoint-a sa unapred definisanim odgovorima, GraphQL najčešće izlaže jedan endpoint, a klijent u telu zahteva navodi koje podatke želi da dobije. Server zatim izvršava upit u odnosu na definisanu šemu i vraća odgovor koji po strukturi odgovara traženom selection set-u.

Osnovna ideja GraphQL-a jeste da klijent ne mora da bude ograničen fiksnim oblikom odgovora koji je server unapred definisao za određeni endpoint. Umesto toga, server definiše skup tipova, polja i operacija koje su dostupne, dok klijent bira konkretna polja koja su mu potrebna u datom trenutku. Ovo je naročito korisno u aplikacijama sa bogatim korisničkim interfejsima, gde različiti ekrani često zahtevaju različite kombinacije podataka.

GraphQL se često opisuje kao query language za API-je, ali je važno razumeti da on nije samo jezik upita. On podrazumeva i runtime za izvršavanje tih upita, tip sistem, pravila validacije, introspection mehanizam i obrazac povezivanja polja sa resolver funkcijama. U .NET ekosistemu, jedna od najpoznatijih i najzrelijih implementacija GraphQL servera jeste HotChocolate, koji je korišćen u praktičnom delu ovog rada.

2.4.1. Osnovni koncepti GraphQL-a

Centralni pojam u GraphQL-u jeste šema. Šema opisuje koje tipove sistem izlaže, koja polja ti tipovi imaju, koji upiti i mutacije postoje i koje argumente prihvataju. Ona predstavlja formalni ugovor između servera i klijenta. Klijent ne može proizvoljno tražiti bilo koje podatke, već samo ono što je izloženo kroz šemu. Time se zadržava kontrola na strani servera, ali se istovremeno klijentu daje fleksibilnost u izboru konkretnih polja.

GraphQL operacije najčešće se dele na query i mutation. Query se koristi za čitanje podataka, dok se mutation koristi za operacije koje menjaju stanje sistema. Na primer,

query može dohvatiti proizvod po identifikatoru, listu proizvoda ili detalje porudžbine. Mutation može kreirati novu porudžbinu, promeniti status ili izvršiti neku drugu poslovnu operaciju. Iako GraphQL koristi najčešće isti HTTP endpoint, razlika između čitanja i izmene stanja postoji na nivou GraphQL operacija.

Resolver predstavlja funkciju ili metod koji zna kako da obezbedi vrednost za određeno polje. Kada klijent pošalje GraphQL upit, runtime analizira selection set i poziva odgovarajuće resolvere kako bi sastavio rezultat. U jednostavnim slučajevima resolver može direktno dohvatiti podatke iz baze. U složenijim slučajevima može pozvati application handler, koristiti DataLoader, izvršiti autorizaciju ili kombinovati podatke iz više izvora. Ovaj model je veoma fleksibilan, ali zahteva pažljivo projektovanje kako ne bi doveo do prevelikog broja poziva ili neefikasnih upita.

Selection set je deo GraphQL upita u kome klijent navodi koja polja želi. Na primer, klijent može tražiti samo naziv i cenu proizvoda, bez opisa, SKU vrednosti ili kategorije. Server zatim vraća JSON odgovor koji sadrži samo tražena polja. Ova osobina GraphQL-a direktno rešava over-fetching problem koji se često javlja kod REST-a. Ako klijentu treba mali podskup podataka, on može poslati uski upit i dobiti manji odgovor.

GraphQL podržava i argumente, filtriranje, sortiranje i paginaciju, u zavisnosti od implementacije i konfiguracije servera. U HotChocolate biblioteci postoje atributi i middleware komponente koje omogućavaju projektovanje, filtriranje i sortiranje nad IQueryable izvorima podataka. Ovo može značajno pojednostaviti implementaciju određenih query scenarija, ali takođe zahteva razumevanje načina na koji se GraphQL upiti prevode u upite prema bazi.

2.4.2. GraphQL kao klijentski definisan upitni model

Najvažnija razlika između REST-a i GraphQL-a jeste u tome ko definiše oblik odgovora. Kod REST-a, oblik odgovora definiše server kroz endpoint i DTO model. Kod GraphQL-a, server definiše šemu, ali klijent bira konkretna polja iz te šeme. Ova razlika značajno menja odnos između frontend i backend dela sistema.

U REST pristupu, ako frontend timu treba novi oblik podataka, često je potrebno da backend tim doda novi endpoint ili proširi postojeći response model. Kod GraphQL-a, ako potrebna polja već postoje u šemi, frontend tim može samostalno promeniti upit i dobiti drugačiji oblik odgovora bez izmene backend koda. To može ubrzati razvoj klijentskih aplikacija, naročito kada više timova radi nad istim backend sistemom.

Ova fleksibilnost ne znači da GraphQL uklanja potrebu za dobrim API dizajnom. Naprotiv, GraphQL šema mora biti pažljivo projektovana jer ona postaje centralni ugovor sistema. Loše projektovana šema može dovesti do konfuznih upita, slabih performansi i teškog održavanja. Ako se šema previše direktno oslanja na strukturu baze podataka, promene u bazi mogu uticati na API. Ako je šema previše apstraktna, može postati teška za korišćenje. Zbog toga dobar GraphQL API zahteva pažljivo balansiranje između domenskog modela, potreba klijenata i performansnih ograničenja.

GraphQL takođe omogućava prirodno dohvaćanje povezanih podataka. Na primer, klijent može u jednom upitu zatražiti porudžbinu, kupca, stavke porudžbine i proizvode. U REST svetu bi za to bio potreban namenski endpoint ili više zasebnih poziva. Ipak, ova

fleksibilnost ima cenu. Svako dodatno povezano polje može zahtevati dodatnu obradu na serveru. Ako resolveri nisu optimizovani, može doći do N+1 problema, gde se za listu entiteta izvršava veliki broj dodatnih upita prema bazi. Zbog toga se u GraphQL implementacijama često koriste DataLoader-i i projekcije kako bi se povezani podaci dohvatili efikasnije.

Klijentski definisan upitni model posebno je koristan u aplikacijama sa više različitih klijenata. Web aplikacija može tražiti širi skup polja za veliki ekran, mobilna aplikacija užu skup polja radi uštede protoka, dok administrativni panel može tražiti dodatne interne informacije. Ako svi koriste istu GraphQL šemu, svaki klijent može oblikovati upit prema svojim potrebama. Ovo je jedna od glavnih praktičnih prednosti GraphQL pristupa.

2.4.3. Prednosti GraphQL pristupa

Glavna prednost GraphQL-a jeste smanjenje over-fetching-a. Klijent ne mora da dobije ceo unapred definisan DTO ako mu je potreban samo deo podataka. U praktičnom delu ovog rada ova prednost se eksplicitno meri kroz scenario u kome REST vraća puni proizvod, dok GraphQL traži samo naziv i cenu. Takav scenario pokazuje jednu od najvažnijih situacija u kojoj GraphQL može imati i manji payload i bolju latenciju, jer se zaista obrađuje i prenosi manje podataka.

Druga prednost jeste smanjenje under-fetching-a. Klijent može u jednom upitu zatražiti povezane podatke, umesto da šalje više zasebnih REST zahteva. Na primer, umesto da prvo zatraži porudžbinu, zatim kupca, zatim proizvode, može u jednom GraphQL upitu navesti celu potrebnu strukturu. Ovo može pojednostaviti klijentski kod i smanjiti broj mrežnih round-trip poziva, naročito u realnim mrežnim uslovima gde latencija između klijenta i servera nije zanemarljiva.

Treća prednost jeste strogo tipizirana šema. GraphQL šema omogućava klijentima i alatima da znaju koje operacije postoje, koja polja su dostupna i koji tipovi se očekuju. Introspection omogućava razvojne alate koji mogu automatski prikazati dokumentaciju, predlagati polja i validirati upite. Ovo može značajno poboljšati developer experience, naročito u većim timovima i sistemima sa kompleksnim API-jem.

GraphQL je takođe pogodan za evoluciju API-ja bez klasičnog verzionisanja endpoint-a. U REST-u se često uvode verzije kao što su `/api/v1` i `/api/v2`, dok GraphQL češće koristi postepeno dodavanje novih polja i označavanje starih polja kao deprecated. Klijenti mogu nastaviti da koriste postojeća polja dok se ne prebace na nova. Ovaj model ne uklanja potrebu za disciplinom, ali omogućava drugačiji pristup evoluciji API ugovora.

Još jedna prednost jeste jedinstvena ulazna tačka za različite upite. Umesto velikog broja endpoint-a, GraphQL server izlaže jednu šemu i jedan endpoint. Ovo može pojednostaviti određene aspekte klijentske integracije, jer se sva komunikacija odvija kroz isti mehanizam. Međutim, ovo istovremeno znači da se praćenje, autorizacija, keširanje i analiza zahteva moraju raditi na nivou GraphQL operacija i polja, a ne samo na nivou HTTP ruta.

2.4.4. Ograničenja GraphQL pristupa

Iako GraphQL donosi značajnu fleksibilnost, on uvodi i dodatnu složenost. Prvo ograničenje jeste veći execution overhead u odnosu na jednostavne REST endpoint-e. GraphQL server mora da primi query, parsira ga, validira u odnosu na šemu, napravi plan izvršavanja, pozove odgovarajuće resolvere i sastavi odgovor u obliku koji odgovara selection set-u. Kod jednostavnih upita, ovaj dodatni trošak može biti veći od koristi koju donosi fleksibilnost. U praktičnim merenjima ovog rada, to se vidi u scenarijima gde oba pristupa vraćaju isti fiksni skup polja, a REST ostvaruje nižu latenciju.

Drugo ograničenje je potencijalni N+1 problem. Pošto GraphQL omogućava prirodno kretanje kroz povezane podatke, lako se može dogoditi da se za listu roditeljskih entiteta izvrši dodatni upit za svako dete. Na primer, ako se dohvata 50 proizvoda i za svaki proizvod posebno učitava kategorija, broj upita može značajno porasti. Ovaj problem se rešava DataLoader-ima, batchovanjem i projekcijama, ali to zahteva dodatno znanje i pažljivo podešavanje.

Treće ograničenje odnosi se na keširanje. REST se prirodno uklapa sa HTTP keširanjem, jer različiti resursi imaju različite URL-ove i GET zahteve. Kod GraphQL-a se veliki broj različitih upita šalje na isti endpoint, najčešće kao POST zahtev. Zbog toga standardno HTTP keširanje nije dovoljno i često su potrebni dodatni mehanizmi, kao što su persisted queries, client-side caching ili specijalizovani GraphQL cache slojevi. To ne znači da je GraphQL nemoguće keširati, ali znači da je strategija keširanja složenija.

Četvrto ograničenje je kontrola kompleksnosti upita. Pošto klijent ima veću slobodu, server mora da se zaštiti od previše složenih, dubokih ili skupih upita. U suprotnom, jedan zahtev može izazvati veliku količinu obrade na serveru. Zato ozbiljne GraphQL implementacije često uvode ograničenja dubine upita, kompleksnosti, maksimalnog broja vraćenih elemenata, obaveznu paginaciju i monitoring skupih operacija. Kod REST-a je ovaj problem često jednostavniji, jer svaki endpoint ima unapred poznat oblik i očekivanu cenu izvršavanja.

Peto ograničenje odnosi se na observability i operativnu preglednost. Kod REST-a je često dovoljno posmatrati HTTP metodu i putanju da bi se razumelo koja operacija se izvršava. Kod GraphQL-a svi zahtevi idu na isti endpoint, pa je potrebno pratiti naziv operacije, selection set, trajanje pojedinačnih resolvera i eventualne greške po poljima. To zahteva prilagođeno logovanje i metrike kako bi se dobila ista operativna vidljivost koju REST prirodno pruža kroz rute.

GraphQL, dakle, nije univerzalno bolje rešenje, već pristup koji rešava određene probleme uz cenu dodatne složenosti. Njegova vrednost je najveća kada klijenti zaista koriste fleksibilnost selection set-a, kada postoji više različitih potrošača API-ja ili kada je potrebno smanjiti broj round-trip zahteva. U sistemima sa jednostavnim, stabilnim i unapred poznatim response modelima, REST često ostaje jednostavniji i efikasniji izbor.

2.5. Komparativni pregled REST i GraphQL pristupa

REST i GraphQL predstavljaju dva različita modela izgradnje Web API sistema. Oba pristupa mogu se koristiti za iste poslovne domene i oba mogu biti implementirana u .NET okruženju, ali se značajno razlikuju u načinu na koji organizuju komunikaciju između klijenta i servera. REST stavlja fokus na resurse i endpoint-e, dok GraphQL stavlja fokus na šemu i upite. REST server unapred definiše oblik odgovora, dok GraphQL omogućava klijentu da izabere polja koja želi iz definisane šeme.

Najjednostavnije rečeno, REST je pogodniji kada su potrebe klijenata predvidive, response modeli stabilni, a API operacije jasno mapirane na resurse. GraphQL je pogodniji kada klijenti imaju različite potrebe, kada se često traže različite kombinacije podataka i kada je važno smanjiti over-fetching ili broj mrežnih poziva. Međutim, ova podela nije apsolutna. Dobro projektovan REST API može imati namenske endpoint-e za složene prikaze, dok loše projektovan GraphQL API može imati loše performanse zbog neefikasnih resolvera.

U pogledu performansi, REST često ima prednost u jednostavnim i fiksnim scenarijima. Ako endpoint vraća unapred poznat DTO i koristi optimizovan SQL upit, serverska obrada može biti veoma brza. GraphQL u tim slučajevima ima dodatni trošak parsiranja, validacije i izvršavanja upita. Međutim, kada klijent traži samo mali podskup polja, GraphQL može biti efikasniji jer ne mora da materijalizuje i serijalizuje nepotrebne podatke. To znači da se performanse ne mogu procenjivati samo na osnovu tehnologije, već moraju biti povezane sa konkretnim oblikom zahteva.

U pogledu veličine odgovora, REST najčešće vraća manji overhead kada je response shape dobro prilagođen. JSON odgovor REST endpoint-a obično sadrži direktno objekat ili listu objekata. GraphQL odgovor sadrži dodatni envelope, najčešće kroz *data* objekat, a kod grešaka i dodatna polja. Zbog toga GraphQL može imati veći payload kada vraća isti skup podataka. Međutim, ako GraphQL koristi selection set za dohvaćanje manjeg broja polja, ukupan payload može biti manji od REST odgovora.

U pogledu fleksibilnosti, GraphQL ima jasnu prednost. Klijent može menjati oblik upita bez promene server endpoint-a, pod uslovom da su potrebna polja već dostupna u šemi. REST je manje fleksibilan jer je oblik odgovora unapred definisan. Sa druge strane, upravo ta manja fleksibilnost čini REST jednostavnijim za kontrolu, keširanje, dokumentovanje i operativno praćenje. Kod GraphQL-a se fleksibilnost mora kontrolisati kroz dobar schema design, ograničenja upita, DataLoader-e, projekcije i observability.

U pogledu razvoja i održavanja, izbor zavisi od organizacije tima i prirode aplikacije. Ako jedan backend tim razvija API za jedan stabilan frontend, REST može biti jednostavniji i dovoljan. Ako više frontend timova koristi isti backend i često traži različite oblike podataka, GraphQL može smanjiti broj izmena na backend-u i ubrzati razvoj klijenata. Međutim, GraphQL zahteva više početnog ulaganja u šemu, resolvere, performansnu optimizaciju i sigurnosna ograničenja.

Tabela 1 prikazuje sažet komparativni pregled ova dva pristupa.

Tabela 1. Komparativni pregled REST i GraphQL pristupa

Karakteristika	REST	GraphQL
Osnovni model	Resursi i endpoint-i	Šema, tipovi i upiti
Broj endpoint-a	Više endpoint-a	Najčešće jedan endpoint
Oblik odgovora	Definiše server	Definiše klijent kroz selection set
Transport	Najčešće HTTP sa različitim metodama	Najčešće HTTP POST prema <i>/graphql</i>
Tipična operacija čitanja	<i>GET /resource/{id}</i>	<i>query { resource(id) { fields } }</i>
Operacije izmene	POST, PUT, PATCH, DELETE	Mutation
Over-fetching	Moguć	Smanjen kada klijent traži samo potrebna polja
Under-fetching	Moguć, često se rešava dodatnim endpoint-ima	Smanjen kroz ugnježdene upite
Keširanje	Prirodnije kroz HTTP mehanizme	Složenije, često traži dodatne strategije
Dokumentovanje	OpenAPI, Swagger	Schema introspection
Server-side kompleksnost	Uglavnom niža	Uglavnom viša
Fleksibilnost klijenta	Manja	Veća
Rizik od skupih upita	Lakše kontrolisan po endpoint-u	Potrebna kontrola dubine i kompleksnosti
Tipična prednost	Jednostavnost i performanse fixed-shape odgovora	Fleksibilnost i selektivno dohvaćanje podataka

Iz ove tabele se vidi da REST i GraphQL ne treba posmatrati kao direktne zamene u svakom mogućem slučaju. Oni imaju različite prednosti i različite troškove. REST nudi jednostavniji, eksplicitniji i često efikasniji model kada su zahtevi stabilni. GraphQL nudi fleksibilniji model kada klijenti imaju promenljive potrebe i kada je važno precizno kontrolisati oblik odgovora.

Za potrebe ovog rada posebno je važno da se teorijska poređenja provere kroz praktičnu implementaciju. Teorijski se može očekivati da REST bude efikasniji u jednostavnim i unapred definisanim scenarijima, dok GraphQL treba da pokaže prednost u slučaju selektivnog dohvaćanja podataka. Međutim, stvarni rezultati zavise od implementacije, korišćenih biblioteka, načina pristupa bazi i oblika konkretnih upita. Zbog toga je u nastavku rada opisan tehnološki okvir i praktična aplikacija CommerceHub, na osnovu kojih su izvedena benchmark merenja.

3. Tehnološki okvir rada

3.1. .NET platforma i ASP.NET Core

.NET platforma predstavlja savremeni razvojni ekosistem za izgradnju različitih tipova aplikacija, uključujući web aplikacije, Web API sisteme, desktop aplikacije, cloud servise, pozadinske radne procese i distribuirane sisteme. U kontekstu ovog rada, najvažniji deo .NET ekosistema jeste ASP.NET Core, framework namenjen razvoju web aplikacija i HTTP API-ja. ASP.NET Core je pogodan za ovakav rad jer kombinuje visoke performanse, stabilan model razvoja, ugrađenu dependency injection podršku, fleksibilan middleware pipeline i dobru integraciju sa modernim alatima za observability i deployment.

Jedna od važnih osobina ASP.NET Core platforme jeste modularnost. Aplikacija se sastoji od niza middleware komponenti koje obrađuju HTTP zahtev. Zahtev prolazi kroz pipeline, gde se mogu izvršavati različiti zadaci kao što su rutiranje, autentifikacija, autorizacija, logovanje, obrada grešaka i mapiranje endpoint-a. Ovakav model omogućava da se aplikacija prilagodi konkretnim potrebama bez nepotrebnog opterećenja komponentama koje nisu potrebne.

ASP.NET Core takođe nudi ugrađeni dependency injection kontejner. To znači da se servisi, handler-i, DbContext, konfiguracije i druge zavisnosti registruju centralno, a zatim se automatski ubrizgavaju u endpoint-e, klase i druge komponente. U praktičnom delu rada ovo je važno jer REST endpoint-i i GraphQL resolveri mogu koristiti iste application handler-e i isti persistence sloj. Time se postiže doslednost implementacije i smanjuje dupliranje koda.

U pogledu performansi, ASP.NET Core je veoma relevantan za razvoj Web API sistema jer je optimizovan za obradu velikog broja HTTP zahteva. Minimalni overhead framework-a, podrška za asinhrono programiranje i efikasna serijalizacija JSON odgovora čine ga pogodnim za benchmark analizu. Pošto se u ovom radu porede dva API pristupa u istom runtime okruženju, .NET platforma obezbeđuje zajedničku osnovu za oba testa.

U praktičnom sistemu CommerceHub, .NET se koristi za implementaciju svih glavnih delova aplikacije: domenskog sloja, application sloja, infrastrukture, REST API-ja, GraphQL API-ja, migration servisa, Aspire AppHost projekta i benchmark projekta. Takva organizacija omogućava da ceo sistem bude tehnološki konzistentan, a da se razlike između REST i GraphQL pristupa posmatraju na nivou API sloja i načina izvršavanja zahteva.

ASP.NET Core je takođe pogodan za poređenje REST i GraphQL pristupa zato što podržava oba modela. REST se može implementirati direktno kroz Minimal APIs, dok se GraphQL može dodati kroz HotChocolate biblioteku. Oba API-ja mogu raditi u odvojenim web projektima, ali koristiti iste zajedničke biblioteke za domensku i aplikacionu logiku. Upravo takva struktura korišćena je u praktičnom delu rada.

3.2. ASP.NET Core Minimal APIs

ASP.NET Core Minimal APIs predstavljaju pristup definisanju HTTP API endpoint-a sa minimalnom količinom ceremonijalnog koda. Za razliku od tradicionalnog pristupa sa kontrolerima, atributima i action metodama, Minimal APIs omogućavaju da se endpoint definiše direktno mapiranjem HTTP metode i rute na funkciju koja obrađuje zahtev. Ovakav pristup je uveden kako bi se pojednostavila izgradnja manjih i srednjih API servisa, ali je vremenom postao dovoljno zreo i za ozbiljnije aplikacije koje žele eksplicitniji i lakši API sloj.

U praktičnom delu rada, REST API je implementiran upravo korišćenjem Minimal APIs pristupa. Svaki endpoint mapira jednu HTTP operaciju i prosleđuje zahtev odgovarajućem application handler-u. Na taj način REST sloj ostaje tanak i nema sopstvenu poslovnu logiku. Njegova odgovornost je da primi HTTP zahtev, izdvoji parametre, pozove odgovarajući use case i vrati HTTP odgovor.

Primer pojednostavljenog REST endpoint-a može izgledati ovako:

Listing 1. Primer REST endpoint-a

```
app.MapGet("products/{id}", async (
    Guid id,
    IQueryHandler<GetProductByIdQuery, ProductResponse> handler,
    CancellationToken cancellationToken) =>
{
    Result<ProductResponse> result = await handler.Handle(
        new GetProductByIdQuery(id),
        cancellationToken);

    return result.Match(
        onSuccess: product => Results.Ok(product),
        onFailure: CustomResults.Problem);
});
```

Ovaj primer pokazuje nekoliko važnih osobina implementacije. Prvo, endpoint direktno prima id iz rute. Drugo, application handler se dobija kroz dependency injection. Treće, endpoint ne zna detalje pristupa bazi podataka. Četvrto, rezultat se mapira na odgovarajući HTTP odgovor. Ovakva organizacija omogućava jasan tok zahteva i olakšava poređenje sa GraphQL pristupom, gde resolver ili query metoda ima sličnu ulogu posrednika između API sloja i application logike.

Minimal APIs dobro odgovaraju ciljevima ovog rada jer ne uvode dodatnu složenost tradicionalnih kontrolera. Pošto je fokus rada na poređenju REST i GraphQL pristupa, REST implementacija treba da bude jednostavna, moderna i efikasna. Minimal APIs omogućavaju upravo to: definisanje eksplicitnih endpoint-a bez nepotrebnog šablonskog koda. Time se smanjuje verovatnoća da rezultati benchmark-a budu posledica framework ceremonije, a ne samog API pristupa.

Još jedna prednost Minimal APIs pristupa jeste dobra integracija sa OpenAPI dokumentacijom, dependency injection sistemom, middleware pipeline-om i standardnim ASP.NET Core mehanizmima. Endpoint-i se mogu grupisati, označiti tagovima, validirati i dokumentovati. U radu to nije centralna tema, ali je važno za praktičnu upotrebljivost implementirane aplikacije.

U kontekstu REST i GraphQL poređenja, Minimal APIs predstavljaju dobar izbor za REST stranu jer nude savremenu i performantnu implementaciju REST endpoint-a. Poređenje GraphQL-a sa zastarelom ili prekomerno složenom REST implementacijom ne bi bilo fer. Korišćenjem Minimal APIs pristupa REST strana dobija modernu i efikasnu osnovu, što rezultate čini relevantnijim.

3.3. HotChocolate GraphQL

HotChocolate je GraphQL server biblioteka za .NET ekosistem. Ona omogućava izgradnju GraphQL API-ja nad .NET aplikacijama, definisanje query i mutation tipova, rad sa šemom, filtriranje, sortiranje, projekcije, DataLoader mehanizam i integraciju sa ASP.NET Core pipeline-om. U praktičnom delu ovog rada HotChocolate se koristi za implementaciju GraphQL API sloja sistema CommerceHub.

U osnovnom obliku, GraphQL API u HotChocolate biblioteci definiše se kroz klase koje predstavljaju Query, Mutation i eventualno dodatne tipove. Query klasa sadrži metode za čitanje podataka, dok Mutation klasa sadrži metode za operacije koje menjaju stanje sistema. HotChocolate na osnovu tih klasa i konfiguracije generiše GraphQL šemu koju klijenti mogu istraživati i koristiti.

Primer pojednostavljene query metode može izgledati ovako:

Listing 2. Primer GraphQL query metode

```
[UseProjection]
[UseFiltering]
[UseSorting]
public IQueryable<Product> GetProducts(
    [Service] IApplicationDbContext dbContext) =>
    dbContext.Products.AsNoTracking();
```

Ovaj primer prikazuje nekoliko važnih HotChocolate mogućnosti. Atribut *UseProjection* omogućava da GraphQL selection set utiče na to koja polja se projektuju iz izvora podataka. *UseFiltering* omogućava generisanje filter argumenata, dok *UseSorting* omogućava sortiranje. Kada se koristi zajedno sa Entity Framework Core i *IQueryable*, HotChocolate može deo GraphQL upita prevesti u efikasniji upit prema bazi podataka, umesto da se svi entiteti najpre materijalizuju u memoriji.

U ovom radu posebno je važna mogućnost server-side projekcije. Bez projekcije, GraphQL bi mogao dohvatati cele entitete čak i kada klijent traži samo nekoliko polja. To bi poređenje učinilo manje fer, jer bi GraphQL bio kažnjen lošom implementacijom, a ne

samom paradigmatom. Korišćenjem projekcija GraphQL implementacija dobija mogućnost da zaista iskoristi selection set i smanji nepotreban rad u scenarijima gde klijent traži mali broj polja.

HotChocolate podržava i DataLoader mehanizam, koji se koristi za rešavanje N+1 problema. DataLoader omogućava da se više pojedinačnih zahteva za povezanim podacima grupiše u jedan batch upit. Na primer, ako više proizvoda zahteva svoje kategorije, umesto da se za svaki proizvod posebno učitava kategorija, DataLoader može prikupiti sve potrebne identifikatore i izvršiti jedan upit. Ovo je veoma važan koncept u GraphQL implementacijama jer se ugnježdjeni selection set-ovi lako mogu pretvoriti u veliki broj pojedinačnih pristupa bazi.

Međutim, HotChocolate i GraphQL generalno uvode dodatni execution pipeline. Svaki GraphQL zahtev mora biti parsiran, validiran, povezan sa šemom i izvršen kroz resolvere. Ovaj overhead je očekivan i predstavlja cenu fleksibilnosti. Upravo zato je u benchmark scenarijima važno razlikovati situacije u kojima GraphQL koristi svoju fleksibilnost od situacija u kojima vraća isti fiksni skup podataka kao REST. Kada klijent traži isti kompletan shape, REST često ima manji overhead. Kada klijent traži samo nekoliko polja, GraphQL može biti efikasniji jer radi manje posla.

U praktičnom sistemu CommerceHub, HotChocolate se koristi kao reprezentativna i savremena GraphQL implementacija za .NET. Time GraphQL strana poređenja nije zasnovana na ručno pisanom ili nestandardnom rešenju, već na biblioteci koja podržava tipične produkcijske koncepte kao što su šema, projekcije, filtering, sorting i DataLoader-i.

3.4. Entity Framework Core i PostgreSQL

Entity Framework Core predstavlja objektno-relacioni mapper za .NET platformu koji omogućava rad sa relacionim bazama podataka korišćenjem .NET klase i LINQ upita. U praktičnom delu ovog rada EF Core se koristi kao zajednički persistence mehanizam za REST i GraphQL API sloj. To znači da oba pristupa čitaju i upisuju podatke kroz isti model podataka, isti *DbContext* i istu PostgreSQL bazu, čime se postiže bolja uporedivost rezultata.

Uloga Entity Framework Core-a u sistemu CommerceHub nije samo tehnička. On predstavlja važan deo metodologije poređenja. Kada bi REST API koristio jedan način pristupa podacima, a GraphQL API drugi, bilo bi teško zaključiti da li razlike u performansama potiču od API paradigme ili od različitog persistence sloja. Zbog toga se oba API-ja oslanjaju na isti infrastructure sloj i istu bazu podataka. Na taj način se razlike u rezultatima mogu u većoj meri povezati sa načinom izlaganja podataka kroz REST ili GraphQL, a manje sa razlikama u samom skladištenju podataka.

EF Core omogućava nekoliko različitih načina dohvaćanja podataka. Jedan pristup je korišćenje *Include* i *ThenInclude* metoda za učitavanje povezanih entiteta. Ovaj pristup je pogodan kada server unapred zna koji objektni graf treba da vrati. Na primer, kod dohvaćanja porudžbine sa kupcem, adresama, stavkama porudžbine i proizvodima, REST

endpoint može koristiti unapred definisan query koji uključuje sve potrebne relacije. Takav pristup je jasan i efikasan za fiksne oblike odgovora.

Drugi pristup je projekcija kroz *Select*, gde se iz baze direktno čita samo onaj skup kolona koji je potreban za određeni response model. Ovo je naročito važno za performanse jer sprečava nepotrebno učitavanje celih entiteta. U REST implementaciji, response DTO modeli često se formiraju upravo kroz projekciju. Na primer, lista proizvoda može vratiti samo identifikator, naziv, SKU, opis, cenu i naziv kategorije, bez učitavanja svih navigacionih svojstava proizvoda. Takva projekcija smanjuje količinu podataka koji se materijalizuju u memoriji i doprinosi boljim performansama.

Kod GraphQL implementacije EF Core ima posebnu ulogu zato što HotChocolate može koristiti *IQueryable* i server-side projekcije. Kada GraphQL upit traži samo određena polja, middleware za projekciju može uticati na to da se iz baze čitaju samo potrebne kolone. Ovo je veoma važno za scenario minimalnog dohvaćanja podataka, gde GraphQL treba da pokaže prednost u odnosu na REST koji vraća pun DTO. Bez pravilno podešenih projekcija, GraphQL bi mogao izgubiti deo svoje prednosti jer bi server i dalje učitavao nepotrebne podatke.

PostgreSQL je izabran kao relacionala baza podataka. U sistemu CommerceHub koristi se kao zajedničko skladište za sve entitete: proizvode, kategorije, kupce, porudžbine, stavke porudžbina, zalihe i dobavljače. PostgreSQL je pogodan izbor za ovakav rad jer je zrela, stabilna i široko korišćena relacionala baza, dobro podržana u .NET ekosistemu kroz EF Core provider. Takođe se lako pokreće u Docker kontejneru, što omogućava ponovljivo lokalno okruženje za razvoj i testiranje.

Važno je istaći da se baza ne koristi samo kao sporedni deo aplikacije, već kao deo eksperimentalnog okruženja. Benchmark rezultati zavise od toga kako API sloj pristupa bazi, koliko efikasno formira upite, da li učitava nepotrebne podatke i koliko često pristupa povezanim entitetima. Zato je za fer poređenje bilo važno da oba API pristupa rade nad istim datasetom i istom bazom. Na taj način REST i GraphQL scenariji ne mere različite podatke, već različite načine dolaska do istog ili ekvivalentnog rezultata.

U radu se koristi dataset srednje veličine, dovoljno velik da bude relevantniji od trivijalnog primera, ali i dalje dovoljno mali da se može stabilno izvršavati u lokalnom Docker okruženju. Time se postiže ravnoteža između praktičnosti i merljivosti. Premali dataset ne bi dovoljno pokazao razlike u listama, projekcijama i ugnježđenim podacima, dok bi prevelik dataset otežao lokalno pokretanje, resetovanje baze i ponavljanje benchmark testova.

3.5. .NET Aspire i observability podrška

.NET Aspire se u ovom radu koristi kao pomoćni tehnološki okvir za lokalnu orkestraciju i posmatranje aplikacije. Njegova uloga nije centralna u smislu istraživačkog pitanja, jer se rad pre svega bavi poređenjem REST i GraphQL pristupa, ali je veoma korisna za praktičnu realizaciju eksperimenta. Aspire omogućava da se više delova sistema definiše i pokrene iz jednog mesta: PostgreSQL baza, migration service, REST API, GraphQL API i prateći dashboard.

U savremenom razvoju backend sistema često nije dovoljno pokrenuti samo jednu aplikaciju. Potrebno je pokrenuti bazu podataka, eventualne pomoćne servise, više API procesa i alate za posmatranje rada sistema. Ručno pokretanje svih ovih komponenti može biti nepraktično i sklono greškama. Aspire rešava ovaj problem tako što omogućava da se topologija lokalnog sistema opiše kroz AppHost projekat. Time se dobija kontrolisan način pokretanja svih zavisnosti i aplikacionih servisa.

U sistemu CommerceHub, Aspire AppHost pokreće PostgreSQL kontejner, zatim migration service koji primenjuje migracije i seeduje bazu, a nakon toga pokreće REST i GraphQL API. Ovakav redosled je važan zato što oba API-ja treba da rade nad već pripremljenom bazom podataka. Kada bi se API servisi pokrenuli pre završetka migracija ili seedovanja, benchmark testovi mogli bi biti nestabilni ili neuporedivi. Aspire omogućava da se kroz zavisnosti između resursa definiše da se API servisi pokreću tek kada je baza spremna i kada je migration service završio svoj posao.

Pored orkestracije, Aspire pruža i dashboard za posmatranje sistema. Kroz njega se mogu pratiti servisi, njihove zavisnosti, logovi, osnovne metrike i tragovi izvršavanja. Iako detaljna observability analiza nije glavni predmet ovog rada, ova mogućnost je korisna tokom razvoja i testiranja. Na primer, ukoliko benchmark pokaže neočekivano veliku latenciju, logovi i osnovni telemetry podaci mogu pomoći da se utvrdi da li problem dolazi iz API sloja, baze podataka, inicijalizacije aplikacije ili nekog drugog dela sistema.

Observability je u ovom radu prisutan kao podržavajući koncept, u meri u kojoj doprinosi stabilnoj i ponovljivoj praktičnoj implementaciji. Za benchmark poređenje posebno je važno da okruženje bude što konzistentnije. Ako se REST API pokreće u jednom okruženju, a GraphQL API u drugom, rezultati bi mogli biti pristrasni. Aspire omogućava da se oba API-ja pokrenu pod istom lokalnom topologijom, sa istom bazom i istim infrastrukturnim uslovima. Naravno, lokalno okruženje ne može u potpunosti zameniti produkciono okruženje, ali za potrebe akademskog rada pruža dovoljno kontrolisanu osnovu za poređenje.

3.6. NBomber kao alat za benchmark testiranje

NBomber je alat za load testing i benchmark testiranje pisan za .NET ekosistem. U ovom radu koristi se za eksperimentalno poređenje REST i GraphQL API implementacija. Njegova osnovna uloga je da generiše kontrolisano opterećenje prema API endpoint-ima i meri različite performansne metrike, kao što su broj uspešnih zahteva, latencija, percentili, maksimalno vreme odgovora i količina prenetih podataka.

Benchmark testiranje je neophodno zato što teorijska analiza sama po sebi nije dovoljna za objektivno poređenje REST i GraphQL pristupa. Na osnovu teorije može se očekivati da REST ima manji overhead kod jednostavnih i fiksnih odgovora, dok GraphQL može imati prednost kada klijent traži samo podskup polja. Međutim, stvarni rezultati zavise od implementacije, korišćenih biblioteka, načina pristupa bazi, veličine odgovora i runtime okruženja. NBomber omogućava da se ova očekivanja provere kroz konkretne merljive scenarije.

U praktičnom delu rada definisano je pet benchmark scenarija. Svaki scenario ima REST i GraphQL varijantu. REST varijanta šalje HTTP zahtev prema odgovarajućem REST endpoint-u, dok GraphQL varijanta šalje GraphQL upit ili mutaciju prema */graphql* endpoint-u. Za read scenarije koristi se isti load profile: faza postepenog povećanja opterećenja i faza stabilnog opterećenja. Za scenario koji uključuje upis u bazu koristi se blaži profil opterećenja, jer operacije upisa imaju veći uticaj na bazu podataka.

Važan deo metodologije jeste warm-up faza. Prvih nekoliko zahteva nakon pokretanja aplikacije često uključuje dodatne troškove kao što su JIT kompilacija, inicijalizacija EF Core modela, otvaranje konekcija ka bazi, inicijalizacija GraphQL šeme i priprema internih runtime struktura. Ako bi se ti zahtevi uključili u konačne rezultate, benchmark bi delimično merio hladan start umesto stabilnog ponašanja sistema. Zato se u ovom radu koristi odbačena warm-up faza pre merenja, kako bi rezultati bolje odražavali steady-state ponašanje.

NBomber rezultati se analiziraju kroz više metrika. Prosečna latencija daje opštu sliku, ali sama po sebi nije dovoljna. Zbog toga se posebno posmatraju percentili p50, p95 i p99. P50 pokazuje tipično vreme odgovora, p95 pokazuje ponašanje sporijih pet procenata zahteva, dok p99 ukazuje na ekstremniji tail latency. Ove metrike su važne zato što korisnici često primećuju upravo sporije zahteve, a ne samo prosečno ponašanje sistema.

Pored latencije, meri se i veličina odgovora po zahtevu. Ovo je posebno relevantno za REST i GraphQL poređenje. REST često ima manji JSON overhead kada vraća isti skup podataka, dok GraphQL može imati manji ukupan payload ako klijent traži samo nekoliko polja. Upravo zato payload analiza predstavlja važan deo rezultata. Ona pokazuje ne samo koliko brzo server odgovara, već i koliko podataka se prenosi između servera i klijenta.

Korišćenjem NBomber-a praktični deo rada dobija eksperimentalnu osnovu. Umesto da se zaključci zasnivaju samo na opštim očekivanjima, oni se izvode na osnovu konkretnih merenja. To ne znači da su rezultati univerzalni za sve REST i GraphQL sisteme, ali znači da su validni za opisanu implementaciju, dataset i testno okruženje. Upravo takav pristup odgovara cilju rada: komparativnoj analizi zasnovanoj na istoj poslovnoj osnovi i merljivim rezultatima.

4. Projektovanje praktičnog sistema CommerceHub

4.1. Namena praktičnog sistema

Praktični deo rada realizovan je kroz aplikaciju CommerceHub. Reč je o sistemu za naručivanje proizvoda koji služi kao eksperimentalna osnova za poređenje REST i GraphQL pristupa u .NET okruženju. Aplikacija nije zamišljena kao kompletan produkcion e-commerce sistem, već kao pažljivo ograničen poslovni domen koji sadrži dovoljno entiteta, relacija i operacija da omogući smisleno poređenje Web API pristupa.

Glavna namena sistema CommerceHub jeste da obezbedi isti poslovni kontekst za dve različite API implementacije. REST API i GraphQL API izlažu funkcionalnosti nad istim

domenom, koriste istu bazu podataka i oslanjaju se na isti application sloj. Na taj način se dobija kontrolisano okruženje u kome se mogu analizirati razlike koje nastaju zbog API paradigme i načina izvršavanja zahteva, a ne zbog potpuno različite poslovne logike.

Domen naručivanja proizvoda izabran je iz više razloga. Prvo, lako je razumljiv i ne zahteva posebno stručno znanje da bi se shvatile osnovne operacije. Proizvodi, kupci, porudžbine i stavke porudžbina predstavljaju pojmove koji su poznati i intuitivni. Drugo, domen prirodno sadrži povezane podatke. Porudžbina pripada kupcu, porudžbina sadrži stavke, stavke se odnose na proizvode, proizvodi pripadaju kategorijama, a zalihe proizvoda povezane su sa dobavljačima. Ovakva struktura je pogodna za poređenje REST i GraphQL pristupa jer omogućava i jednostavne i ugnježdene upite.

Treće, domen omogućava definisanje različitih benchmark scenarija. Jednostavan proizvod može se koristiti za merenje osnovnog overhead-a. Porudžbina sa povezanim podacima može se koristiti za testiranje dubokog grafa. Proizvod sa mnogo polja može se koristiti za analizu over-fetching-a. Lista proizvoda može se koristiti za testiranje paginacije i serijalizacije većeg broja objekata. Kreiranje i naknadno čitanje porudžbine omogućava testiranje kombinovanog write-read scenarija.

CommerceHub je zato projektovan kao praktičan, ali kontrolisan sistem. On nije preopterećen dodatnim funkcionalnostima kao što su autentifikacija, autorizacija, plaćanja, notifikacije, messaging ili keširanje, jer bi te funkcionalnosti proširile scope rada i otežale izolovano poređenje REST i GraphQL pristupa. Umesto toga, sistem je fokusiran na ono što je najvažnije za temu rada: oblik API zahteva i odgovora, pristup podacima, veličinu payload-a i performanse pod opterećenjem.

4.2. Poslovni domen

Poslovni domen sistema CommerceHub zasniva se na pojednostavljenom modelu online prodavnice. Centralni entiteti su proizvodi, kategorije, kupci, porudžbine, stavke porudžbina, zalihe i dobavljači. Iako je domen pojednostavljen, on sadrži dovoljno relacija da omogući realistične scenarije čitanja i pisanja podataka.

Entitet *Product* predstavlja proizvod koji se može naručiti. On sadrži osnovne informacije kao što su identifikator, naziv, SKU oznaka, opis, cena i veza ka kategoriji. Proizvod je jedan od najvažnijih entiteta u benchmark scenarijima, jer se koristi u jednostavnom čitanju, minimalnom dohvatanju podataka, listama i stavkama porudžbina. Upravo zbog toga je model proizvoda pogodan za poređenje over-fetching i minimal-fetch scenarija.

Entitet *Category* predstavlja kategoriju proizvoda. Kategorije mogu biti organizovane hijerarhijski, jer kategorija može imati roditeljsku kategoriju. Ova struktura omogućava da proizvodi budu grupisani u logične celine. U API odgovorima kategorija se često prikazuje kroz naziv ili kraći objekat sa identifikatorom i nazivom. Veza između proizvoda i kategorije važna je za poređenje REST i GraphQL pristupa jer pokazuje kako oba API-ja rade sa povezanim podacima.

Entitet *Customer* predstavlja kupca. Kupac ima osnovne podatke kao što su ime i email adresa, kao i jednu ili više adresa. U datasetu svaki kupac ima jednu adresu, što je

dovoljno za scenarije koji uključuju porudžbine i podatke o kupcu. Kupac je povezan sa porudžbinama, jer svaka porudžbina pripada određenom kupcu.

Entitet *Order* predstavlja porudžbinu. Porudžbina sadrži identifikator, kupca, status, datum kreiranja, ukupan iznos i kolekciju stavki porudžbine. Ovaj entitet je posebno važan za deep graph scenario, jer dohvaćanje porudžbine često podrazumeva i dohvaćanje kupca, stavki porudžbine, proizvoda i njihovih kategorija. Takav objektni graf omogućava poređenje načina na koji REST i GraphQL rade sa ugnježđenim podacima.

Entitet *OrderLine* predstavlja jednu stavku porudžbine. Stavka sadrži proizvod, količinu i cenu proizvoda u trenutku poručivanja. Čuvanje cene u stavci porudžbine je važno jer se cena proizvoda može promeniti nakon kreiranja porudžbine, ali istorijska vrednost porudžbine treba da ostane ista. Ovaj detalj pokazuje da domen nije samo skup tehničkih tabela, već ima i osnovnu poslovnu logiku.

Entitet *StockItem* predstavlja stanje zaliha određenog proizvoda kod određenog dobavljača. On povezuje proizvod i dobavljača i sadrži podatke kao što su količina na stanju i nivo za ponovno naručivanje. Ovaj entitet omogućava modelovanje dodatnih relacija oko proizvoda. Iako nije centralan u svim benchmark scenarijima, koristan je za potencijalne scenarije koji uključuju dublje povezane podatke i rizik od N+1 problema.

Entitet *Supplier* predstavlja dobavljača. Dobavljač ima naziv i kontakt informacije. Kroz vezu sa zalihama i potencijalnim supply order entitetima, dobavljači proširuju domen proizvoda i čine model realističnijim. Ipak, u okviru trenutnog benchmark poređenja fokus nije na složenim procesima nabavke, već na API pristupu podacima.

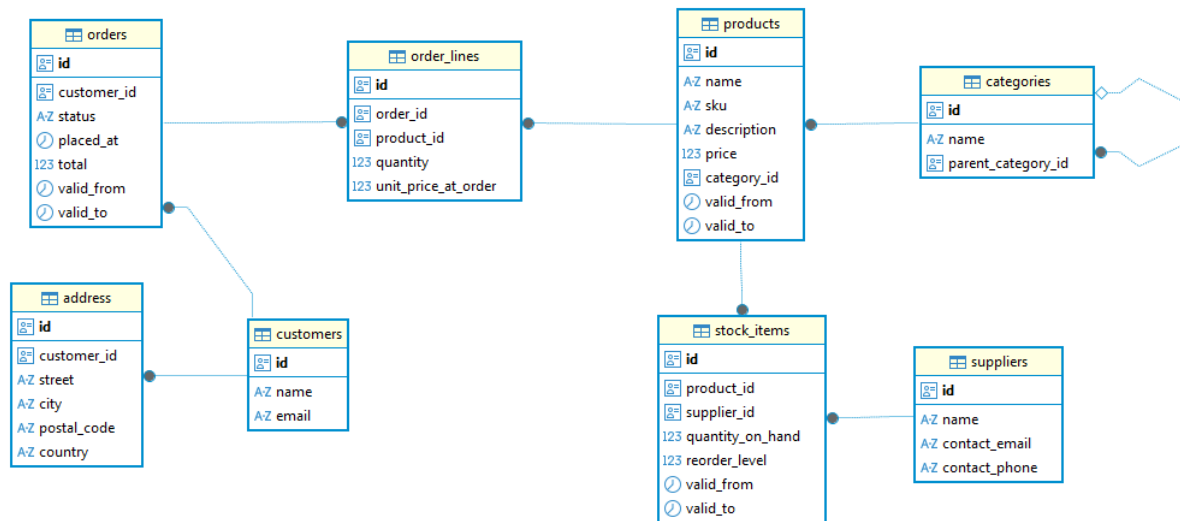
Domen se može posmatrati kroz nekoliko tipičnih tokova. Jedan tok je pregled proizvoda i njihovih kategorija. Drugi tok je kreiranje porudžbine za određenog kupca. Treći tok je dohvaćanje detalja porudžbine sa povezanim informacijama. Ovi tokovi su dovoljno reprezentativni da pokažu razlike između REST i GraphQL pristupa bez nepotrebnog proširivanja sistema.

4.3. Model baze podataka

Model baze podataka sistema CommerceHub izveden je iz opisanog poslovnog domena. Baza sadrži tabele za kupce, adrese, proizvode, kategorije, porudžbine, stavke porudžbina, zalihe, dobavljače i povezane entitete nabavke. Odnosi između tabela prate prirodne poslovne veze. Kupac može imati više adresa i više porudžbina. Porudžbina ima više stavki. Stavka porudžbine referencira proizvod. Proizvod pripada kategoriji i može imati više zapisa o zalihama. Zaliha povezuje proizvod i dobavljača.

Ovakav relacioni model pogodan je za poređenje REST i GraphQL pristupa jer sadrži i jednostavne i složene upite. Jednostavno čitanje proizvoda zahteva pristup jednoj tabeli i eventualno povezanoj kategoriji. Dohvaćanje porudžbine sa detaljima zahteva čitanje više povezanih tabela. Lista proizvoda zahteva paginaciju, sortiranje i projekciju. Minimalno dohvaćanje podataka zahteva samo podskup kolona. Sve ove situacije predstavljaju realne obrasce korišćenja Web API sistema.

Slika 1. ER dijagram CommerceHub domena



Još jedna važna osobina modela jeste da se koristi isti database schema za oba API pristupa. REST i GraphQL ne koriste odvojene baze, odvojene tabele ili odvojene modele podataka. Time se izbegava mogućnost da jedan pristup dobije prednost zbog drugačije strukture baze. Svi benchmark scenariji izvršavaju se nad istim podacima, što doprinosi metodološkoj ispravnosti poređenja.

4.4. Arhitektura rešenja

Rešenje CommerceHub organizovano je kroz više projekata, pri čemu svaki projekat ima jasno definisanu odgovornost. Ovakva organizacija omogućava razdvajanje poslovnog domena, application logike, infrastrukture, API slojeva i benchmark testova. Time se postiže čitljivija struktura i smanjuje sprega između delova sistema.

Osnovni projekti u rešenju su:

- Domain
- Application
- Infrastructure
- Web.RestApi
- Web.GraphQLApi
- Web.MigrationService
- Aspire.AppHost
- Benchmarks

Domain projekat sadrži domenske entitete i osnovna poslovna pravila. U njemu se nalaze klase kao što su *Product*, *Category*, *Customer*, *Order*, *OrderLine*, *StockItem* i *Supplier*. Ovaj sloj ne zavisi od konkretnog API pristupa, baze podataka ili framework-a za izlaganje endpoint-a. Njegova uloga je da predstavi poslovni model sistema.

Application projekat sadrži use case logiku, query i command objekte, handler-e i response modele. Ovaj sloj posreduje između API slojeva i infrastrukture. REST endpoint-i i GraphQL resolveri pozivaju application handler-e kada treba da izvrše određenu poslovnu operaciju ili dohvate podatke. Time se izbegava dupliranje poslovne logike u REST i GraphQL slojevima.

Infrastructure projekat sadrži implementaciju pristupa bazi podataka, EF Core *DbContext*, konfiguracije entiteta, migracije i data seeding logiku. Ovaj sloj je zadužen za komunikaciju sa PostgreSQL bazom i pripremu početnog dataseta. Pošto oba API sloja koriste isti infrastructure sloj, poređenje se vrši nad istom persistence osnovom.

Web.RestApi projekat predstavlja REST API implementaciju. On sadrži Minimal API endpoint-e koji izlažu funkcionalnosti sistema kroz HTTP rute. Endpoint-i su tanki i uglavnom prosleđuju zahteve application handler-ima. Njihova odgovornost je HTTP komunikacija, a ne poslovna logika.

Web.GraphQLApi projekat predstavlja GraphQL API implementaciju. On sadrži GraphQL query i mutation tipove, kao i konfiguraciju HotChocolate servera. GraphQL sloj izlaže šemu kroz koju klijenti mogu da šalju upite i mutacije. Kao i REST sloj, on se oslanja na isti application i infrastructure sloj.

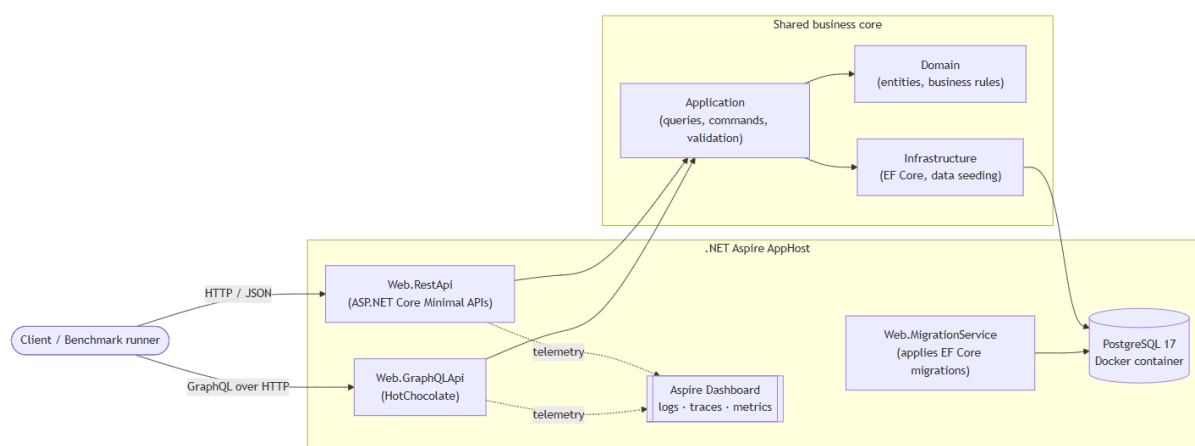
Web.MigrationService projekat zadužen je za primenu migracija i seedovanje baze prilikom pokretanja sistema. Njegova uloga je da baza bude spremna pre nego što REST i GraphQL API počnu da obrađuju zahteve. Ovo je naročito važno za benchmark testove, jer svi scenariji moraju raditi nad poznatim i pripremljenim datasetom.

Aspire.AppHost projekat opisuje lokalnu topologiju sistema. Kroz njega se definiše PostgreSQL kontejner, migration service, REST API, GraphQL API i njihove međusobne zavisnosti. On omogućava pokretanje celog sistema iz jednog mesta.

Benchmarks projekat sadrži NBomber scenarije za merenje performansi. On šalje zahteve REST i GraphQL API-ju, meri rezultate i generiše izveštaje. Odvajanje benchmark koda u poseban projekat čini eksperimentalni deo jasnijim i ponovljivijim.

Ovakva arhitektura može se prikazati dijagramom.

Slika 2. Arhitektura CommerceHub sistema



Dijagram prikazuje benchmark runner kao klijenta koji šalje zahteve REST i GraphQL API-ju. Oba API-ja supovezana sa Application slojem, Application sa Domain i

Infrastructure slojem, a Infrastructure sa PostgreSQL bazom. Aspire AppHost prikazan je kao okvir koji pokreće PostgreSQL, migration service i oba API servisa.

Najvažnija arhitektonska odluka jeste da REST i GraphQL budu dva različita API sloja nad istim core-om. Time se izbegava situacija u kojoj su to dve različite aplikacije koje samo slučajno imaju sličan domen. U CommerceHub sistemu one su dva načina izlaganja iste poslovne logike.

4.5. Zajednički poslovni sloj

Zajednički poslovni sloj predstavlja osnovu fer poređenja između REST i GraphQL pristupa. Ako bi REST API imao jednu implementaciju poslovnih pravila, a GraphQL drugu, rezultati benchmark-a ne bi mogli da se tumače kao razlika između API paradigmi. Zbog toga je u sistemu CommerceHub poslovna logika izdvojena u zajedničke projekte koje koriste oba API sloja.

Application sloj sadrži query i command handler-e. Query handler-i se koriste za operacije čitanja, kao što su dohvaćanje proizvoda, liste proizvoda ili porudžbine. Command handler-i se koriste za operacije koje menjaju stanje, kao što je kreiranje nove porudžbine. Ovi handler-i rade sa *IApplicationDbContext* interfejsom, što omogućava da API slojevi ne zavise direktno od konkretne implementacije baze.

REST endpoint tipično kreira query ili command objekat i prosleđuje ga odgovarajućem handler-u. GraphQL resolver ili mutation metoda može uraditi isto. Time se postiže da oba API pristupa koriste isti tok obrade poslovne operacije. Razlika je u tome kako zahtev dolazi do application sloja i kako se rezultat vraća klijentu.

Na primer, operacija kreiranja porudžbine može se pozvati kroz REST kao *POST /orders*, dok se u GraphQL-u može pozvati kroz *placeOrder* mutaciju. Iako su ulazni API oblici različiti, sama poslovna logika kreiranja porudžbine treba da bude ista. To uključuje proveru proizvoda, formiranje stavki porudžbine, računanje ukupne vrednosti i čuvanje porudžbine u bazi.

Zajednički poslovni sloj pomaže i u održavanju sistema. Ako se promeni poslovno pravilo, ono se menja na jednom mestu, a ne posebno u REST i GraphQL implementaciji. To je dobra praksa i van konteksta benchmark-a, ali je ovde posebno korisna jer čuva integritet poređenja.

Naravno, nije svaka operacija potpuno identična na nivou API sloja. REST i GraphQL imaju različite načine oblikovanja odgovora. REST vraća unapred definisan DTO, dok GraphQL može vratiti izbor polja koji klijent traži. Međutim, kada se porede scenariji, response shape je usklađen koliko je moguće, tako da oba pristupa vraćaju isti ili ekvivalentan skup podataka. Time se smanjuje rizik da jedan pristup bude brži samo zato što vraća manje podataka.

Zajednički poslovni sloj je zato jedna od najvažnijih projektantskih odluka u praktičnom delu rada. On omogućava da se CommerceHub koristi kao kontrolisano eksperimentalno okruženje, a ne samo kao obična demonstraciona aplikacija.

4.6. Data seeding

Da bi benchmark testiranje bilo ponovljivo, sistem mora imati poznat i stabilan skup podataka. Zbog toga CommerceHub koristi data seeding, odnosno automatsko popunjavanje baze podacima prilikom inicijalnog pokretanja. Seed podaci se generišu kroz C# kod i upisuju u PostgreSQL bazu preko Entity Framework Core *DbContext*-a.

Tabela 2 prikazuje približan obim seedovanih podataka.

Tabela 2. Dataset korišćen u praktičnom delu rada

Entitet	Približan broj zapisa	Napomena
Categories	20	Više glavnih kategorija i potkategorija
Suppliers	25	Generisani dobavljači sa kontakt podacima
Products	1.000	Proizvodi raspoređeni kroz različite kategorije
StockItems	~1.667	Svaki proizvod ima najmanje jedan zapis zaliha
Customers	500	Svaki kupac ima jednu adresu
Orders	5.000	Porudžbine raspoređene kroz kupce
OrderLines	~17.500	Svaka porudžbina ima 2–5 stavki

Ovaj dataset omogućava da se testiraju različiti obrasci pristupa podacima. Lista proizvoda nije trivijalna, jer postoji 1.000 proizvoda. Porudžbine su dovoljno brojne da write-read scenario ne radi nad praznim ili gotovo praznim sistemom. Stavke porudžbina omogućavaju smisleno testiranje ugnježdenih struktura. Zalihe i dobavljači proširuju domen i ostavljaju prostor za dodatne scenarije.

Data seeding se izvršava samo ako baza već nema podatke. To je važno jer omogućava da benchmark testovi rade nad stabilnim baseline datasetom. Ako bi se podaci dodavali pri svakom pokretanju, rezultati bi se menjali iz testa u test. Sa druge strane, kada je potrebno promeniti veličinu dataseta ili ponovo izvršiti seedovanje, baza se može resetovati, nakon čega migration service ponovo kreira šemu i popunjava podatke.

Kod seedovanja treba voditi računa da podaci budu deterministički u dovoljnoj meri. To znači da struktura dataseta treba da bude stabilna: isti broj proizvoda, kupaca, porudžbina i povezanih entiteta. Potpuna realističnost podataka nije glavni cilj, jer benchmark ne meri poslovnu raznovrsnost naziva i opisa, već ponašanje API-ja nad poznatom količinom podataka i relacija. Ipak, podaci su organizovani tako da liče na realan domen i da proizvodi budu raspoređeni po kategorijama i dobavljačima.

Data seeding je važan i za izbor identifikatora koji se koriste u benchmark scenarijima. Pre pokretanja testova potrebno je znati identifikator proizvoda, porudžbine ili kupca koji se koristi u zahtevima. Benchmark projekat može automatski otkriti potrebne identifikatore kroz početne REST ili GraphQL pozive, ili ih može dobiti kroz environment promenljive. Time se izbegava ručno menjanje benchmark koda nakon svakog resetovanja baze.

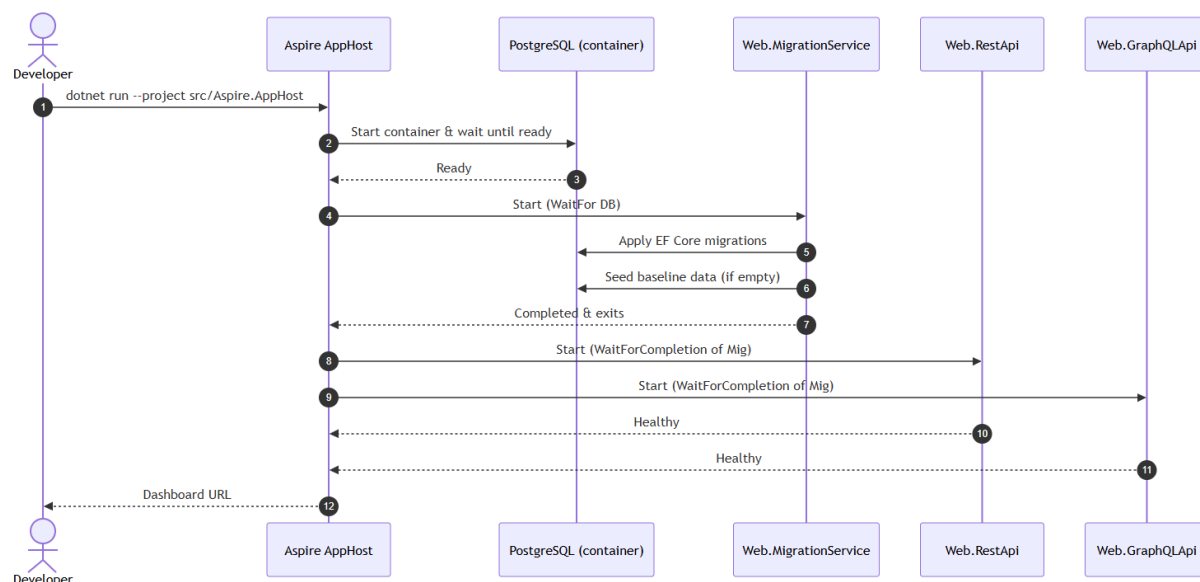
4.7. Pokretanje sistema kroz .NET Aspire

Pokretanje sistema CommerceHub organizovano je kroz .NET Aspire AppHost projekat. Aspire omogućava da se svi potrebni delovi sistema pokrenu kao povezana celina. Umesto ručnog pokretanja PostgreSQL baze, zatim migracija, zatim REST API-ja, pa GraphQL API-ja, AppHost definiše resurse i zavisnosti između njih.

Tipična sekvenca pokretanja izgleda ovako. Najpre se pokreće PostgreSQL kontejner. Nakon što baza postane dostupna, pokreće se migration service. On primenjuje EF Core migracije i izvršava data seeding ako je baza prazna. Tek nakon završetka migration service-a pokreću se REST API i GraphQL API. Na taj način oba API-ja počinju rad nad istom pripremljenom bazom.

Ovaj proces može se prikazati sekvencijalnim dijagramom.

Slika 3. Sekvenca pokretanja sistema kroz .NET Aspire



Tok pokretanja može se opisati sledećim koracima:

- Developer pokreće Aspire AppHost.
- AppHost pokreće PostgreSQL kontejner.
- AppHost čeka da PostgreSQL bude spreman.
- Pokreće se MigrationService.
- MigrationService primenjuje migracije.
- MigrationService seeduje bazu ako je prazna.
- Nakon završetka migracija i seedovanja pokreću se REST i GraphQL API.
- Aspire Dashboard prikazuje servise, logove i osnovne telemetry podatke.

Ovakvo pokretanje ima više prednosti. Prva prednost je ponovljivost. Svaki put kada se sistem pokrene kroz AppHost, koristi se ista definisana topologija. Druga prednost je manja mogućnost greške, jer se zavisnosti pokreću ispravnim redosledom.

Treća prednost je preglednost, jer se kroz Aspire Dashboard može videti koji servisi rade i kako su povezani.

Za benchmark testiranje ova organizacija je posebno korisna. REST i GraphQL API se pokreću kao odvojeni servisi, ali pod istim lokalnim uslovima. Oba koriste istu bazu i isti seedovani dataset. Load generator zatim šalje zahteve na njihove konkretne URL adrese. Pošto Aspire može dodeliti portove dinamički, benchmark projekat koristi environment promenljive za REST i GraphQL base URL vrednosti.

Važno je naglasiti da Aspire u ovom radu nije predmet performansnog poređenja. On se ne poredi sa drugim orkestracionim alatima, niti je cilj rada analiza Aspire platforme. Njegova uloga je pomoćna: da obezbedi kontrolisano lokalno okruženje za razvoj, pokretanje i osnovno posmatranje sistema. Time se omogućava da fokus rada ostane na REST i GraphQL pristupu.

5. Implementacija REST i GraphQL API slojeva

Nakon definisanja domena, arhitekture rešenja i tehnološkog okvira, sledeći korak predstavlja implementacija dva API sloja nad istim poslovnim sistemom. U ovom poglavlju prikazani su najvažniji elementi implementacije oba API sloja. Fokus nije na prikazu celokupnog izvornog koda, već na onim delovima koji su važni za razumevanje poređenja: način definisanja endpoint-a, način pozivanja application sloja, oblikovanje odgovora, korišćenje GraphQL selection set-a i usklađivanje REST i GraphQL scenarija za potrebe benchmark testiranja.

5.1. Implementacija REST API-ja

REST API u sistemu CommerceHub implementiran je kao poseban ASP.NET Core projekat. Njegova odgovornost je da izloži funkcionalnosti sistema kroz HTTP endpoint-e. Svaki endpoint odgovara određenoj operaciji nad domenom, kao što su dohvaćanje proizvoda, listanje proizvoda, dohvaćanje porudžbine ili kreiranje nove porudžbine.

Tipični REST endpoint-i u aplikaciji su:

- *GET /products/{id}*
- *GET /products/list?page=1&pageSize=50*
- *GET /orders/{id}*
- *POST /orders*

Ovakav skup endpoint-a odražava resursno orijentisan model REST pristupa. Proizvodi i porudžbine predstavljaju resurse, dok HTTP metode određuju tip operacije. *GET* zahtevi koriste se za čitanje podataka, dok se *POST* koristi za kreiranje nove porudžbine. Query parametri, kao što su *page* i *pageSize*, koriste se za paginaciju liste proizvoda.

REST endpoint-i su projektovani kao tanki sloj iznad application logike. To znači da endpoint ne sadrži složenu poslovnu logiku, već prima zahtev, izdvaja parametre, kreira odgovarajući query ili command objekat i prosleđuje ga handler-u. Handler zatim izvršava poslovnu operaciju ili upit nad bazom podataka, a rezultat se vraća endpoint-u koji ga prevodi u HTTP odgovor.

Ovakva organizacija ima više prednosti. Prvo, REST sloj ostaje jednostavan i pregledan. Drugo, poslovna logika nije vezana za HTTP detalje. Treće, isti application handler-i mogu se koristiti i iz GraphQL sloja. Četvrto, testiranje i održavanje postaju lakši jer se use case logika nalazi u odvojenim klasama.

5.2. Primer REST operacije za čitanje proizvoda

Operacija čitanja proizvoda predstavlja najjednostavniji benchmark scenario. Cilj ovog scenarija je da se izmeri osnovni trošak jednog API zahteva kada se dohvata jedan entitet sa unapred definisanim skupom polja. REST endpoint za ovu operaciju vraća proizvod u obliku DTO modela koji sadrži osnovne informacije potrebne klijentu.

Response model za proizvod može se predstaviti sledećim record tipom:

Listing 3. Response model proizvoda

```
public sealed record ProductResponse(  
    Guid Id,  
    string Name,  
    string Sku,  
    string Description,  
    decimal Price,  
    Guid CategoryId,  
    string CategoryName);
```

Ovaj model predstavlja ugovor REST API-ja prema klijentu. On ne mora biti identičan domenskom entitetu Product. Umesto navigacionog svojstva ka kategoriji, response model sadrži CategoryId i CategoryName, što je klijentu jednostavnije za korišćenje. Ovakvo oblikovanje odgovora je uobičajeno u REST API-jima jer omogućava da server precizno kontroliše strukturu podataka koju vraća.

Handler za dohvaćanje proizvoda može koristiti EF Core projekciju kako bi se iz baze učitala samo potrebna polja. Pojednostavljen primer izgleda ovako:

Listing 4. Primer query handler-a za dohvaćanje proizvoda

```

internal sealed class GetProductByIdQueryHandler(IApplicationDbContext
dbContext)
    : IQueryHandler<GetProductByIdQuery, ProductResponse>
{
    public async Task<Result<ProductResponse>> Handle(
        GetProductByIdQuery query,
        CancellationToken cancellationToken)
    {
        ProductResponse? product = await dbContext.Products
            .AsNoTracking()
            .Where(p => p.Id == query.Id)
            .Select(p => new ProductResponse(
                p.Id,
                p.Name,
                p.Sku,
                p.Description,
                p.Price,
                p.CategoryId,
                p.Category.Name))
            .FirstOrDefaultAsync(cancellationToken);

        if (product is null)
        {
            return Result.Failure<ProductResponse>(
                ProductErrors.NotFound(query.Id));
        }

        return product;
    }
}

```

Ovaj primer pokazuje karakterističan način rada REST implementacije u sistemu CommerceHub. Podaci se ne učitavaju kao puni entiteti ako to nije potrebno, već se direktno projektuju u response model. Time se smanjuje količina materijalizovanih podataka i dobija efikasniji SQL upit.

Za potrebe benchmark poređenja, važno je da GraphQL scenario za dohvaćanje proizvoda traži isti skup polja. Ako bi REST vraćao sedam polja, a GraphQL samo dva, poređenje ne bi bilo fer za scenario jednostavnog čitanja. Zbog toga je u prvom benchmark scenariju GraphQL query usklađen sa REST response modelom, tako da oba pristupa vraćaju identične informacije o proizvodu.

REST pristup je u ovakvom scenariju očekivano efikasan. Server unapred zna oblik odgovora, query handler je optimizovan za taj oblik, a JSON odgovor nema dodatni envelope osim samog objekta. Ovaj scenario zato predstavlja dobru osnovu za merenje osnovnog per-request overhead-a između REST i GraphQL pristupa.

5.3. Primer REST operacije za listu proizvoda

Lista proizvoda predstavlja čest scenario u poslovnim aplikacijama. Klijent želi da dobije stranicu proizvoda, najčešće uz paginaciju i eventualno filtriranje.

Jednostavan primer handler-a za listu proizvoda izgleda ovako:

Listing 5. REST handler za listu proizvoda

```
internal sealed class GetProductListQueryHandler(IApplicationDbContext
dbContext)
    : IQueryHandler<GetProductListQuery, List<ProductResponse>>
{
    public async Task<Result<List<ProductResponse>>> Handle(
        GetProductListQuery query,
        CancellationToken cancellationToken)
    {
        IQueryable<Product> productsQuery =
            dbContext.Products.AsNoTracking();

        if (query.CategoryId.HasValue)
        {
            productsQuery = productsQuery
                .Where(p => p.CategoryId == query.CategoryId.Value);
        }

        List<ProductResponse> items = await productsQuery
            .OrderBy(p => p.Name)
            .Skip((query.Page - 1) * query.PageSize)
            .Take(query.PageSize)
            .Select(p => new ProductResponse(
                p.Id,
                p.Name,
                p.Sku,
                p.Description,
                p.Price,
                p.CategoryId,
                p.Category.Name))
            .ToListAsync(cancellationToken);

        return items;
    }
}
```

Ovaj handler prihvata parametre za paginaciju i opciono filtriranje po kategoriji. Zatim formira query nad proizvodima, primenjuje sortiranje, preskakanje zapisa, ograničenje broja rezultata i projekciju u ProductResponse.

REST endpoint koji koristi ovaj handler može izgledati ovako:

Listing 6. REST endpoint za listu proizvoda

```
app.MapGet("products/list", async (
    Guid? categoryId,
    int? page,
    int? pageSize,
    IQueryHandler<GetProductListQuery, List<ProductResponse>> handler,
    CancellationToken cancellationToken) =>
{
    var query = new GetProductListQuery(
        categoryId,
        page ?? 1,
        pageSize ?? 20);

    Result<List<ProductResponse>> result = await handler.Handle(
        query,
        cancellationToken);

    return result.Match(
        onSuccess: products => Results.Ok(products),
        onFailure: CustomResults.Problem);
});
```

Ovaj endpoint vraća 50 proizvoda i određeni (isti) skup polja koji GraphQL vraća u odgovarajućem scenariju. Na taj način se poređenje ne zasniva na različitom obimu podataka ili različitoj semantici odgovora, već na stvarnoj razlici između REST i GraphQL načina obrade zahteva što je u skladu sa ciljem rada – da se ne favorizuje jedan pristup, već da se oba pristupa dovedu u što sličnije uslove.

5.4. Implementacija GraphQL API-ja

GraphQL API u sistemu CommerceHub implementiran je kao poseban ASP.NET Core projekat koji koristi HotChocolate biblioteku. Za razliku od REST API-ja, GraphQL API ne izlaže više ruta za različite resurse, već izlaže GraphQL šemu kroz jedan endpoint. Klijenti šalju query ili mutation operacije, a server na osnovu šeme i selection set-a određuje koje podatke treba vratiti.

Centralni deo GraphQL implementacije čini *Query* klasa. Ona definiše operacije za čitanje podataka, kao što su dohvaćanje proizvoda, liste proizvoda, kupaca i porudžbina. GraphQL API takođe može imati metode koje direktno vraćaju jedan objekat po identifikatoru. Na primer:

Listing 7. Primer GraphQL query metode za dohvaćanje proizvoda po identifikatoru

```
public async Task<Product?> GetProductById(
    Guid id,
    [Service] IApplicationDbContext dbContext,
    CancellationToken cancellationToken) =>
    await dbContext.Products
        .AsNoTracking()
        .Include(p => p.Category)
        .FirstOrDefaultAsync(p => p.Id == id, cancellationToken);
```

Ova metoda omogućava GraphQL klijentu da zatraži proizvod i zatim kroz selection set odredi koja polja želi. Na primer, jedan klijent može tražiti identifikator, naziv, SKU i kategoriju, dok drugi može tražiti samo naziv i cenu. Server izvršava isti root query, ali oblik odgovora zavisi od selection set-a.

Pored query operacija, GraphQL API sadrži i mutation operacije. Mutation se koristi za operacije koje menjaju stanje sistema, kao što je kreiranje nove porudžbine. U benchmark scenariju write-read, GraphQL mutation kreira porudžbinu i vraća njen identifikator, nakon čega se posebnim GraphQL query-jem dohvataju detalji kreirane porudžbine. Ovo je usklađeno sa REST scenarijem, gde se takođe izvršavaju dva poziva: *POST /orders* i zatim *GET /orders/{id}*.

GraphQL implementacija uvodi drugačiji tok izvršavanja u odnosu na REST. Kada stigne zahtev, HotChocolate najpre parsira GraphQL query, validira ga u odnosu na šemu, određuje koja polja treba izvršiti i zatim poziva odgovarajuće resolve ili projekcije. Ovaj proces daje veliku fleksibilnost, ali uvodi dodatni overhead. Upravo taj overhead se meri u scenarijima gde REST i GraphQL vraćaju isti skup podataka.

Sa druge strane, kada klijent traži samo mali broj polja, GraphQL može imati prednost jer ne mora da učita, projektuje i serijalizuje ceo response model. U tom slučaju selection set nije samo sintaksna pogodnost, već direktno utiče na količinu obrađenih i prenetih podataka. Ovo je naročito vidljivo u benchmark scenariju over-fetching vs minimal fetching.

5.5. Primer GraphQL upita za proizvod

GraphQL upit za proizvod po identifikatoru može se napisati tako da traži isti skup polja kao REST endpoint. Na primer:

Listing 8. GraphQL upit za proizvod sa istim poljima kao REST odgovor

```

query GetProduct($id: UUID!) {
  productById(id: $id) {
    id
    name
    sku
    description
    price
    categoryId
    category {
      name
    }
  }
}

```

Ovaj upit traži identifikator, naziv, SKU, opis, cenu, identifikator kategorije i naziv kategorije. To odgovara REST *ProductResponse* modelu. Iako GraphQL struktura odgovora nije potpuno identična REST strukturi, jer je kategorija predstavljena kao ugnježdjeni objekat, logički skup podataka je isti. U benchmark poređenju to je dovoljno da se scenario smatra usklađenim.

Odgovor na ovakav GraphQL upit ima oblik:

```

{
  "data": {
    "productById": {
      "id": "e7f3...",
      "name": "Product 0001",
      "sku": "SKU-000001",
      "description": "Description for product 0001",
      "price": 15.49,
      "categoryId": "a21b...",
      "category": {
        "name": "Laptops"
      }
    }
  }
}

```

U poređenju sa REST odgovorom, GraphQL ima dodatni *data* envelope i ugnježdjeni oblik za kategoriju. Ovaj dodatni envelope je deo standardnog GraphQL response formata. Kod malih odgovora, kao što je jedan proizvod, taj dodatni JSON overhead može relativno značajno povećati payload. Zato se u rezultatima jednostavnog GET scenarija očekuje da REST ima manji response size kada oba pristupa vraćaju ista polja.

Važno je primetiti da GraphQL klijent može promeniti selection set bez promene server endpoint-a. Ako mu ne treba opis proizvoda, može ga izostaviti. Ako mu treba

samo naziv kategorije, ne mora tražiti njen identifikator. Ova fleksibilnost je centralna razlika u odnosu na REST pristup.

5.6. Primer GraphQL upita sa minimalnim skupom polja

Jedna od glavnih prednosti GraphQL-a jeste mogućnost minimalnog dohvaćanja podataka. Ako klijentu trebaju samo naziv i cena proizvoda, GraphQL upit može biti mnogo uži:

Listing 9. GraphQL upit sa minimalnim skupom polja

```
query GetProductMinimal($id: UUID!) {  
  productById(id: $id) {  
    name  
    price  
  }  
}
```

Ovaj upit traži samo dva polja. Server, uz odgovarajuću implementaciju i projekcije, može obraditi manje podataka i vratiti manji JSON odgovor. Odgovor izgleda ovako:

```
{  
  "data": {  
    "productById": {  
      "name": "Product 0001",  
      "price": 15.49  
    }  
  }  
}
```

U REST pristupu, ako postoji samo endpoint *GET /products/{id}* koji vraća pun *ProductResponse*, klijent će dobiti i polja koja mu nisu potrebna. To je tipičan primer over-fetching problema. REST bi mogao rešiti ovaj problem dodavanjem posebnog endpoint-a ili query parametra za izbor polja, ali to nije standardni osnovni model REST API-ja i zahteva dodatni dizajn.

U benchmark scenariju over-fetching vs minimal fetching upravo se meri ova razlika. REST vraća pun DTO proizvoda, dok GraphQL traži samo *name* i *price*. Ovaj scenario nije zamišljen kao potpuno simetrično poređenje istog response shape-a, već kao demonstracija konkretne prednosti GraphQL-a. Cilj je da se pokaže šta se dešava kada klijent zaista koristi mogućnost selektivnog dohvaćanja podataka.

Rezultati ovog scenarija su važni jer pokazuju da GraphQL prednost nije samo teorijska. Kada klijent traži manji skup polja, GraphQL može preneti manje podataka i

izvršiti manje rada na serveru. To ne znači da će GraphQL uvek biti brži, ali pokazuje da njegova fleksibilnost može imati merljiv efekat u odgovarajućem scenariju.

Ovaj primer takođe pokazuje zašto nije dovoljno upoređivati REST i GraphQL samo kroz identične fixed-shape odgovore. Ako se GraphQL uvek primora da vraća isti pun DTO kao REST, ne meri se jedna od njegovih glavnih prednosti. Zbog toga benchmark metodologija u ovom radu sadrži i simetrične scenarije i scenario koji namerno ističe selektivno dohvaćanje podataka.

5.7. Usklađivanje REST i GraphQL odgovora

Jedan od najvažnijih izazova u ovom radu bilo je usklađivanje REST i GraphQL scenarija. Da bi poređenje bilo fer, nije dovoljno da oba API-ja formalno rade nad istim domenom. Potrebno je da u svakom benchmark scenariju vraćaju isti ili jasno definisan ekvivalentan skup podataka. U suprotnom, jedan pristup može izgledati brže samo zato što vraća manje podataka ili radi manje posla.

Tokom razvoja praktičnog dela, pojedini scenariji su morali biti dodatno prilagođeni upravo iz tog razloga. Na primer, scenario liste proizvoda u početnoj verziji nije bio potpuno usklađen. REST je vraćao jednu stranu proizvoda, dok je GraphQL mogao vratiti veći broj proizvoda ili dublji skup povezanih podataka. Takav rezultat ne bi bio validan za zaključak o performansama REST-a i GraphQL-a, jer se ne bi merila ista operacija. Zbog toga je scenario izmenjen tako da oba pristupa vraćaju prvu stranu od 50 proizvoda i isti logički response shape.

Slično tome, REST endpoint za listu proizvoda je prilagođen tako da za benchmark ne vraća dodatni paged wrapper sa ukupnim brojem zapisa, jer GraphQL scenario ne računa total count. Da je REST radio dodatni *CountAsync()*, a GraphQL ne, rezultat bi bio pristrasan. Uklanjanjem tog dodatnog rada iz benchmark endpoint-a oba pristupa mere isti osnovni zadatak: dohvaćanje 50 proizvoda sa istim skupom polja.

Jedini scenario koji namerno nije simetričan u pogledu response shape-a jeste over-fetching vs minimal fetching. Međutim, to je njegova poenta. On meri situaciju u kojoj REST vraća pun DTO jer je endpoint tako definisan, dok GraphQL koristi svoju mogućnost da traži samo potrebna polja. Taj scenario nije dokaz da je GraphQL univerzalno brži, već demonstracija uslova u kojima njegova fleksibilnost donosi praktičnu korist.

Usklađivanje odgovora je ključno za tumačenje rezultata. Kada REST pobedi u fixed-shape scenarijima, to se može povezati sa njegovim jednostavnijim execution modelom. Kada GraphQL pobedi u minimal-fetch scenariju, to se može povezati sa manjim skupom obrađenih i prenetih podataka. Bez ovog usklađivanja, rezultati bi bili teško objašnjivi i mogli bi dovesti do pogrešnih zaključaka.

5.8. Razlike u implementacionom modelu

Iako REST i GraphQL API slojevi koriste isti domen i istu bazu, njihov implementacioni model se značajno razlikuje. REST zahtev prolazi kroz eksplicitnu HTTP rutu, odgovarajući endpoint i application handler. GraphQL zahtev prolazi kroz GraphQL endpoint, parsing, validaciju, execution plan, resolvere i eventualne projekcije ili DataLoader-e.

Pojednostavljen tok REST zahteva može se prikazati ovako:

HTTP zahtev

- ASP.NET Core rutiranje
- Minimal API endpoint
- Application handler
- EF Core upit
- Projekcija u DTO
- JSON odgovor (response)

REST model je linearan i unapred poznat. Za svaki endpoint server zna koji handler se poziva i koji response model se vraća. To omogućava jednostavniju optimizaciju i često manji runtime overhead. Sa druge strane, fleksibilnost je ograničena na ono što endpoint podržava.

Pojednostavljen tok GraphQL zahteva izgleda ovako:

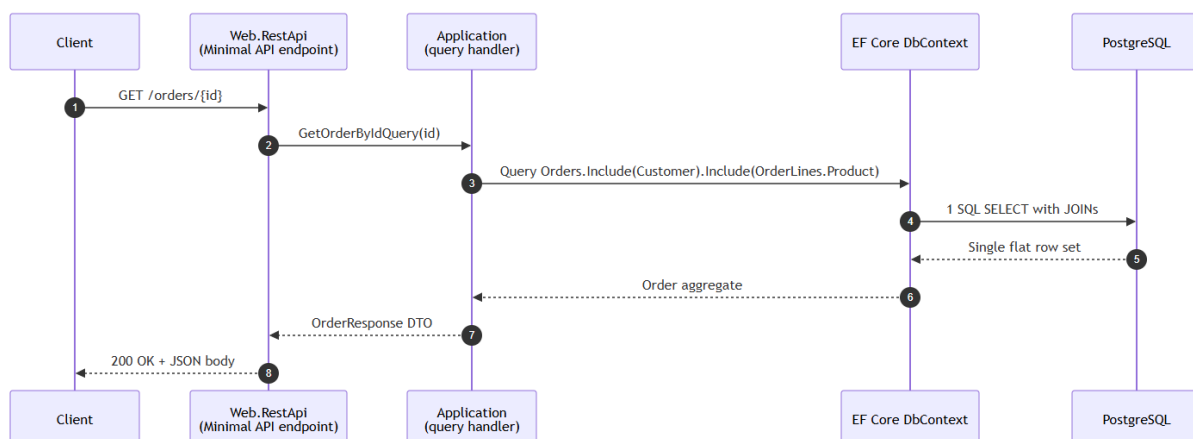
HTTP zahtev ka /graphql endpoint-u

- Parsiranje GraphQL upita
- Validacija u odnosu na šemu
- Planiranje izvršavanja
- Izvršavanje resolvera/projekcija
- EF Core upit
- Oblikovanje GraphQL odgovora (response-a)
- JSON odgovor sa *data* omotačem (response)

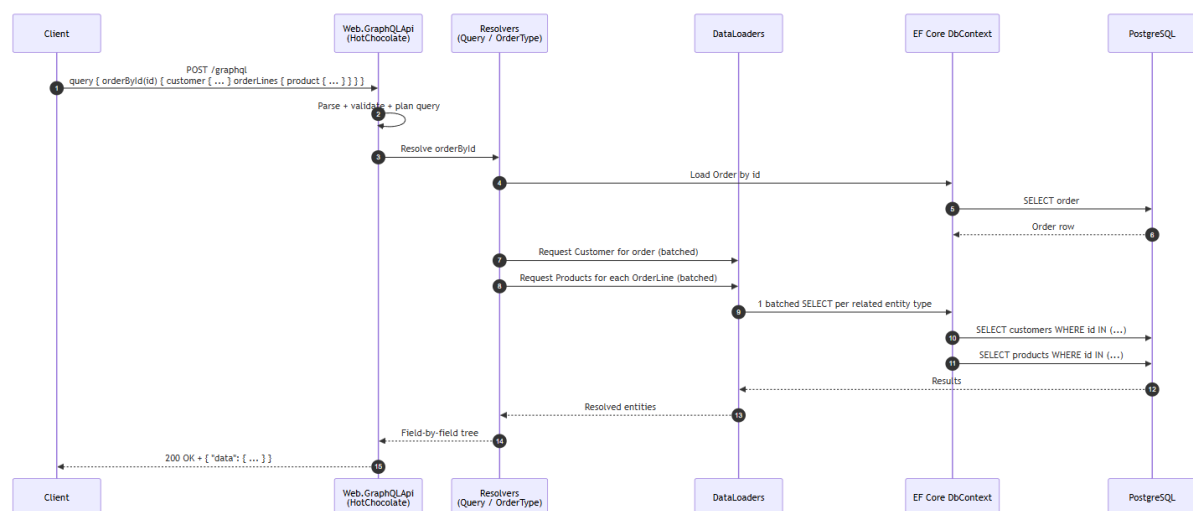
GraphQL model je fleksibilniji, ali složeniji. Server mora obraditi query tekst, proveriti ga u odnosu na šemu i izvršiti samo tražena polja. Ovaj dodatni pipeline ima cenu, ali omogućava klijentu da oblikuje odgovor. Upravo zato GraphQL može biti sporiji kada vraća isti fiksni shape kao REST, ali bolji kada klijent zaista traži manji skup polja.

U radu je korisno prikazati dva sekvencijalna dijagrama.

Slika 4. Tok REST zahteva



Slika 5. Tok GraphQL zahteva



Ovi dijagrami pomažu da se objasne benchmark rezultati. Kada REST ima manju latenciju u jednostavnom GET scenariju, razlog nije samo u tome što je REST „brži“, već u tome što njegov tok izvršavanja ima manje dinamičkih koraka. Kada GraphQL ima prednost u minimal-fetch scenariju, razlog nije samo u tome što je GraphQL „moderniji“, već u tome što selection set omogućava manji obim posla.

Razlika u implementacionom modelu utiče i na operativno praćenje. Kod REST-a je HTTP ruta dovoljna da se zna koja operacija se izvršava. Kod GraphQL-a ista HTTP ruta može nositi veliki broj različitih upita. Zato je za ozbiljan GraphQL sistem korisno logovati naziv operacije, selection set, trajanje resolvera i eventualne greške po poljima. U ovom radu observability nije centralna tema, ali ova razlika pokazuje da API paradigma utiče i na način posmatranja sistema.

Na kraju, može se zaključiti da su REST i GraphQL implementacije u CommerceHub sistemu projektovane tako da budu što uporedivije, ali i da zadrže prirodne osobine svojih pristupa. REST koristi eksplicitne endpoint-e i DTO projekcije.

GraphQL koristi šemu, selection set i projekcije. Upravo to omogućava da benchmark rezultati budu smisleni: oni ne porede dve identične implementacije pod različitim imenima, već dva različita API modela nad istim poslovnim sistemom.

6. Metodologija benchmark testiranja

6.1. Cilj benchmark testiranja

Cilj benchmark testiranja u ovom radu jeste da se REST i GraphQL pristup uporede kroz konkretne, merljive scenarije. Teorijska analiza može objasniti očekivane prednosti i ograničenja oba pristupa, ali tek eksperimentalno merenje pokazuje kako se te razlike odražavaju u konkretnoj implementaciji. Zbog toga benchmark testiranje predstavlja ključni deo praktične evaluacije sistema CommerceHub.

Važno je naglasiti da cilj benchmark-a nije da se dokaže da je jedan pristup univerzalno bolji od drugog. Takav zaključak ne bi bio opravdan, jer performanse zavise od domena, implementacije, veličine podataka, upita, infrastrukture i načina korišćenja. Umesto toga, cilj je da se pokaže kako se REST i GraphQL ponašaju u nekoliko reprezentativnih scenarija i da se rezultati povežu sa teorijskim očekivanjima.

Benchmark testovi su osmišljeni tako da pokriju različite obrasce korišćenja API-ja. Neki scenariji su simetrični, što znači da REST i GraphQL vraćaju isti ili ekvivalentan skup podataka. Takvi scenariji služe za merenje overhead-a i efikasnosti kada oba pristupa rade isti posao. Drugi scenario, over-fetching vs minimal fetching, namerno je asimetričan jer prikazuje situaciju u kojoj GraphQL koristi svoju sposobnost selektivnog dohvaćanja polja.

U merenjima se posmatra više metrika. Latencija pokazuje koliko vremena je potrebno da server obradi zahtev i vrati odgovor. Percentili pokazuju kako se sistem ponaša ne samo u proseku, već i za sporije zahteve. Veličina odgovora pokazuje koliko podataka se prenosi između servera i klijenta. Broj neuspešnih zahteva pokazuje stabilnost sistema pod opterećenjem. Zajedno, ove metrike daju potpuniju sliku od same prosečne brzine.

Posebna pažnja posvećena je fer poređenju. Oba API-ja koriste isti dataset, istu bazu, isti application sloj i isto lokalno runtime okruženje. REST i GraphQL benchmark scenariji izvršavaju se u odvojenim prolazima kako ne bi direktno konkurisali za bazu podataka i load generator u isto vreme. Pre merenja se koristi warm-up faza kako bi se smanjio uticaj hladnog starta. Ova pravila su uvedena da bi rezultati bili što pouzdaniji i lakši za tumačenje.

U nastavku ovog poglavlja opisani su testno okruženje, pravila fer poređenja, merene metrike, load profile i konkretni benchmark scenariji.

6.2. Testno okruženje

Benchmark testiranje sprovedeno je u lokalnom razvojnom okruženju u kome se svi delovi sistema izvršavaju na istoj mašini. To podrazumeva da REST API, GraphQL API, PostgreSQL baza i NBomber load generator rade u okviru istog lokalnog okruženja. Ovakav pristup ne simulira u potpunosti produkcione uslove, jer nema realne mrežne latencije između klijenta i servera, ali omogućava kontrolisanije poređenje samih API implementacija.

Sistem je pokretan kroz .NET Aspire AppHost projekat. Aspire je zadužen za pokretanje PostgreSQL kontejnera, migration service-a, REST API-ja i GraphQL API-ja. Na taj način se obezbeđuje da oba API pristupa rade u istom infrastrukturnom kontekstu. PostgreSQL baza se pokreće kao lokalni Docker kontejner, dok se API aplikacije izvršavaju kao ASP.NET Core procesi. Ovakvo okruženje je pogodno za akademski eksperiment jer omogućava ponovljivo pokretanje i relativno lako resetovanje baze.

REST API implementiran je pomoću ASP.NET Core Minimal APIs tehnologije, dok je GraphQL API implementiran pomoću HotChocolate biblioteke. Oba API-ja koriste isti application sloj i isti persistence sloj zasnovan na Entity Framework Core-u. Baza podataka je PostgreSQL 17, a podaci se automatski seed-uju prilikom inicijalnog pokretanja sistema. Dataset obuhvata proizvode, kategorije, kupce, porudžbine, stavke porudžbina, zalihe i dobavljače.

Za generisanje opterećenja korišćen je NBomber benchmark projekat. NBomber šalje HTTP zahteve prema REST i GraphQL API-ju, meri performanske metrike i generiše izveštaje. REST zahtevi šalju se prema odgovarajućim HTTP rutama, dok GraphQL zahtevi koriste */graphql* endpoint i u telu zahteva sadrže odgovarajući query ili mutation.

Pošto .NET Aspire može dinamički dodeliti portove aplikacijama, benchmark projekat koristi environment promenljive za podešavanje URL adresa REST i GraphQL API-ja. Na taj način isti benchmark kod može raditi nezavisno od konkretnih portova dodeljenih pri pokretanju.

Primer podešavanja URL adresa može izgledati ovako:

```
$env:REST_URL      = "http://localhost:<rest-port>"
$env:GRAPHQL_URL   = "http://localhost:<graphql-port>"

dotnet run --project tests/Benchmarks -c Release
```

Ovakvo podešavanje omogućava da se testovi izvršavaju nad trenutno pokrenutim instancama API-ja. Pre pokretanja benchmark-a potrebno je proveriti da su REST API, GraphQL API i PostgreSQL baza uspešno pokrenuti i da je migration service završio primenu migracija i seedovanje baze.

Važno je naglasiti da su rezultati dobijeni u lokalnom okruženju. To znači da oni dobro prikazuju relativne razlike između dve implementacije u istim uslovima, ali ne predstavljaju apsolutne performanse koje bi se nužno dobile u produkcionom sistemu. U produkciji bi na rezultate uticali mrežna latencija, TLS, reverse proxy, autentifikacija, autorizacija, skaliranje, keširanje, kompresija odgovora i drugi infrastrukturni faktori.

6.3. Pravila fer poređenja

Jedan od najvažnijih delova metodologije jeste definisanje pravila fer poređenja. Poređenje REST i GraphQL pristupa može lako postati pristrasno ako se ne vodi računa o tome da oba pristupa rade isti ili ekvivalentan posao. Na primer, ako REST vraća 50 proizvoda, a GraphQL vraća 1.000 proizvoda sa dodatnim povezanim podacima, razlika u performansama ne bi govorila o kvalitetu API paradigme, već o različitom obimu posla. Zbog toga su benchmark scenariji pažljivo usklađeni.

Prvo pravilo jeste da oba API-ja koriste isti poslovni domen. REST i GraphQL nisu implementirani nad različitim primerima, već nad istim CommerceHub sistemom. Entiteti, odnosi i poslovni tokovi su zajednički. Time se izbegava situacija u kojoj jedan pristup dobija prednost zbog jednostavnijeg ili drugačije modelovanog domena.

Drugo pravilo jeste da oba API-ja koriste istu bazu podataka. Svi podaci nalaze se u istoj PostgreSQL bazi, a oba API-ja im pristupaju kroz isti infrastructure sloj. Ovo je važno jer performanse API-ja u velikoj meri zavise od pristupa podacima. Ako bi jedan API koristio optimizovanu bazu, a drugi ne, poređenje ne bi bilo validno.

Treće pravilo jeste korišćenje istog application sloja. REST endpoint-i i GraphQL resolveri ne sadrže sopstvene, odvojene implementacije poslovne logike, već koriste iste handler-e gde god je to primenljivo. Time se smanjuje dupliranje koda i obezbeđuje da se poredi API sloj, a ne potpuno različite implementacije use case-ova.

Četvrto pravilo jeste usklađivanje response shape-a. U scenarijima gde se meri direktno poređenje performansi, oba pristupa vraćaju isti ili ekvivalentan skup podataka. Na primer, u simple GET scenariju oba API-ja vraćaju ista polja proizvoda. U deep graph scenariju oba vraćaju istu logičku strukturu porudžbine. U paged list scenariju oba vraćaju 50 proizvoda sa istim osnovnim poljima. Ovo pravilo je posebno važno jer veličina odgovora i broj polja direktno utiču na latenciju i payload.

Peto pravilo jeste da se REST i GraphQL testovi izvršavaju u odvojenim pass-ovima. Umesto da se oba API-ja opterećuju istovremeno, najpre se meri jedan pristup, a zatim drugi. Time se izbegava direktna konkurencija za bazu podataka i load generator u istom trenutku. Svaki API dobija sopstveni tok zahteva i sopstveni connection pool. Ovaj pristup čini rezultate stabilnijim, ali uvodi i potencijalni uticaj redosleda izvršavanja, jer drugi test može raditi nad bazom čiji su podaci već delimično zagrejani u buffer/cache memoriji.

Šesto pravilo jeste korišćenje warm-up faze. Pre merenja se izvršava kratka faza zagrevanja koja se ne uključuje u konačne rezultate. Njena svrha je da apsorbuje troškove hladnog starta, JIT kompilacije, inicijalizacije EF Core modela, otvaranja konekcija i pripreme GraphQL šeme. Time se merenje više približava stabilnom ponašanju sistema nakon inicijalnog pokretanja.

Sedmo pravilo jeste da svi scenariji imaju isti load profile, osim scenarija koji uključuje upis u bazu. Read scenariji koriste opterećenje od 20 zahteva u sekundi, dok write-read scenario koristi blaže opterećenje od 5 zahteva u sekundi. Razlog je to što operacije upisa imaju veći uticaj na bazu podataka i ne treba ih opterećivati istim intenzitetom kao čiste read scenarije.

Ova pravila ne uklanjaju sva moguća ograničenja benchmark-a, ali značajno povećavaju njegovu metodološku vrednost. Rezultati se zato mogu tumačiti kao relevantni za konkretnu implementaciju i testno okruženje, uz jasno navedena ograničenja.

6.4. Metrike

Benchmark analiza u ovom radu zasniva se na više metrika. Korišćenje samo jedne metrike, na primer prosečne latencije, ne bi bilo dovoljno za kvalitetno poređenje. API može imati dobar prosek, ali loše ponašanje u sporijim percentilima. Takođe, može biti brz, ali vraćati znatno veći payload. Zbog toga se rezultati analiziraju kroz kombinaciju latencije, percentila, veličine odgovora i broja neuspešnih zahteva.

Prva metrika je *mean latency*, odnosno prosečna latencija. Ona predstavlja aritmetičku sredinu vremena odgovora svih uspešnih zahteva u okviru jednog scenarija. Prosečna latencija je korisna jer daje opšti osećaj o brzini API-ja, ali može biti osetljiva na ekstremne vrednosti. Jedan veoma spor zahtev može povećati prosek, iako je većina zahteva bila brza. Zato prosečna vrednost nikada ne treba da bude jedina osnova za zaključak.

Druga metrika je *p50*, odnosno pedeseti percentil ili medijana. P50 pokazuje vrednost ispod koje se nalazi polovina svih zahteva. Drugim rečima, polovina zahteva bila je brža od *p50* vrednosti, a polovina sporija. Ova metrika često bolje opisuje tipično korisničko iskustvo od proseka, jer nije toliko osetljiva na ekstremne outlier-e.

Treća metrika je *p95*, odnosno devedeset peti percentil. Ona pokazuje vreme ispod kog se nalazi 95% zahteva. Samo najsporijih 5% zahteva ima veću latenciju od *p95* vrednosti. Ova metrika je važna jer pokazuje ponašanje sistema za sporije zahteve. U realnim sistemima korisnici često ne primećuju samo prosečno ponašanje, već i povremene sporije odgovore. Zbog toga je *p95* često važniji od proseka.

Četvrta metrika je *p99*, odnosno devedeset deveti percentil. Ona pokazuje ekstremniji tail latency. Samo najsporijih 1% zahteva ima veću latenciju od ove vrednosti. *P99* je naročito važan kod sistema koji zahtevaju stabilno korisničko iskustvo, jer povremeni veoma spori zahtevi mogu negativno uticati na percepciju performansi. U ovom radu *p99* se koristi da bi se videlo da li neki pristup ima izraženije povremene skokove latencije.

Peta metrika je maksimalna latencija. Ona predstavlja najsporiji zahtev u okviru scenarija. Ova vrednost može biti korisna kao indikator najgoreg slučaja, ali se mora tumačiti oprezno. Jedan izolovan outlier može nastati zbog garbage collection-a, scheduling-a operativnog sistema, privremenog opterećenja računara ili nekog drugog lokalnog efekta. Zato se max vrednost posmatra kao dopunska, a ne glavna metrika.

Šesta metrika je payload per request, odnosno veličina odgovora po zahtevu. Ona pokazuje koliko podataka API vraća klijentu. Ovo je posebno važno u REST i GraphQL poređenju. GraphQL često ima dodatni JSON envelope, ali može smanjiti payload kada klijent traži samo određena polja. REST često ima manji overhead kada vraća isti shape,

ali može vratiti više podataka nego što klijentu treba. Payload zato predstavlja jednu od najvažnijih metrika za procenu praktične efikasnosti API pristupa.

Sedma metrika je fail count, odnosno broj neuspešnih zahteva. U idealnom slučaju, svi zahtevi treba da budu uspešni. Ako se pojave greške, rezultati latencije mogu biti manje relevantni, jer neuspešni zahtevi ne predstavljaju normalan tok obrade. U finalnim benchmark rezultatima svi scenariji završeni su bez neuspešnih zahteva, što znači da su oba API-ja stabilno obradila definisano opterećenje.

Kombinovanjem ovih metrika dobija se potpunija slika performansi. Na primer, API može imati niži p50, ali viši p99, što znači da je tipično brz, ali ima povremene skokove. Drugi API može imati nešto viši p50, ali stabilniji tail. Isto tako, API može biti brži, ali vraćati veći payload. Zato se u analizi rezultata ne posmatra samo jedna vrednost, već odnos više metrika.

6.5. Profil opterećenja

Profil opterećenja definiše način na koji se opterećenje generiše tokom benchmark testa. U ovom radu za read scenarije koristi se isti obrazac opterećenja: najpre faza postepenog povećanja intenziteta, a zatim faza stabilnog opterećenja. Ovakav pristup je realističniji od trenutnog slanja maksimalnog broja zahteva, jer omogućava sistemu da postepeno uđe u opterećenje.

Read scenariji koriste sledeći load profile:

- 20 sekundi ramp-up do 20 zahteva u sekundi
- 40 sekundi stabilnog opterećenja na 20 zahteva u sekundi
- ukupno 60 sekundi merenja
- oko 990 zahteva po API-ju po scenariju

Ramp-up faza znači da se opterećenje postepeno povećava do ciljanog nivoa. Nakon toga sledi hold faza, u kojoj se opterećenje održava na istom nivou. Ukupno trajanje scenarija je 60 sekundi. Ovakav profil omogućava da se posmatra ponašanje sistema pod umerenim, ali stabilnim opterećenjem.

Scenario koji uključuje upis u bazu koristi blaži load profile:

- 20 sekundi ramp-up do 5 zahteva u sekundi
- 40 sekundi stabilnog opterećenja na 5 zahteva u sekundi
- ukupno 60 sekundi merenja
- oko 250 zahteva po API-ju

Razlog za blaže opterećenje jeste činjenica da ovaj scenario ne obuhvata samo čitanje, već i kreiranje nove porudžbine. Operacije upisa uključuju validaciju, formiranje stavki, računanje ukupne vrednosti i upis u bazu. One imaju veći uticaj na bazu podataka nego jednostavni read upiti. Zbog toga bi isti intenzitet kao kod read scenarija mogao promeniti prirodu testa i više meriti granice baze nego razlike između API pristupa.

Pre merenih scenarija izvršava se warm-up faza. Warm-up traje 10 sekundi i njegovi rezultati se ne uključuju u finalne tabele. Cilj warm-up-a je da se aplikacija zagreje i da se smanji uticaj inicijalnih runtime troškova. Bez warm-up-a prvi zahtevi mogu biti sporiji zbog JIT kompilacije, inicijalizacije framework-a, pripreme EF Core modela i otvaranja konekcija. Pošto rad ne analizira cold start, već stabilno ponašanje API-ja, warm-up faza je opravdana.

REST i GraphQL scenariji se pokreću odvojeno. To znači da se, na primer, najpre izvršava REST varijanta scenarija, a zatim GraphQL varijanta, ili obrnuto, u zavisnosti od konfiguracije benchmark runner-a. Ovaj pristup sprečava da jedan API utiče na drugi kroz direktnu konkurenciju za resurse tokom istog testa. Ipak, treba imati u vidu da redosled može uticati na stanje cache-a baze podataka ili operativnog sistema. Zato se ova činjenica navodi kao ograničenje metodologije.

Load profile u ovom radu nije ekstremno. Cilj nije da se pronađe maksimalan throughput koji sistem može izdržati, već da se uporedi relativno ponašanje REST i GraphQL implementacije u kontrolisanim uslovima. Opterećenje od 20 zahteva u sekundi za read scenarije dovoljno je da se uoče razlike u latenciji i payload-u, a da sistem ostane stabilan i bez grešaka.

6.6. Benchmark scenariji

U radu je definisano pet benchmark scenarija. Oni su izabrani tako da pokriju različite obrasce korišćenja API-ja i da istaknu različite osobine REST i GraphQL pristupa. Neki scenariji mere simetrično ponašanje kada oba API-ja vraćaju isti skup podataka, dok jedan scenario posebno meri prednost GraphQL-a u selektivnom dohvatanju polja.

Prvi scenario je **Simple GET**, odnosno dohvatanje jednog proizvoda po identifikatoru. Ovo je najjednostavniji scenario i služi kao osnovni test per-request overhead-a. Oba API-ja vraćaju ista polja proizvoda: identifikator, naziv, SKU, opis, cenu, identifikator kategorije i naziv kategorije. Pošto je poslovna operacija jednostavna, razlike u rezultatu uglavnom potiču od overhead-a API pristupa i strukture odgovora.

Drugi scenario je **Deep graph fetch**, odnosno dohvatanje porudžbine sa povezanim podacima. Oba API-ja vraćaju porudžbinu, podatke o kupcu i stavke porudžbine sa povezanim proizvodima. Ovaj scenario je važan jer se GraphQL često smatra pogodnim za dohvatanje ugnježđenih podataka u jednom zahtevu. Međutim, REST varijanta takođe koristi namenski endpoint koji vraća isti objektni graf u jednom pozivu. Time se poredi dobro projektovan REST endpoint sa GraphQL upitom koji traži isti skup podataka.

Treći scenario je **Over-fetching vs minimal fetching**. U ovom scenariju REST vraća pun response model proizvoda, dok GraphQL traži samo dva polja: naziv i cenu. Ovaj scenario namerno nije potpuno simetričan, jer njegova svrha nije merenje istog response shape-a, već demonstracija jedne od glavnih prednosti GraphQL-a. Ako klijent zaista treba samo mali podskup podataka, GraphQL može smanjiti količinu prenetih i obrađenih informacija.

Četvrti scenario je **Paged product list**. Oba API-ja vraćaju prvu stranu liste proizvoda, sa *page = 1* i *pageSize = 50*. Response shape je usklađen tako da oba pristupa

vraćaju osnovne informacije o proizvodu i kategoriji. Ovaj scenario predstavlja tipičan ekran liste proizvoda u aplikaciji i meri ponašanje API-ja kada se vraća veći broj objekata, ali bez dubljih povezanih struktura.

Peti scenario je **Write + read-back**. U ovom scenariju svaka iteracija najpre kreira novu porudžbinu, a zatim dohvata kreiranu porudžbinu nazad. REST to radi kroz *POST /orders* i *GET /orders/{id}*, dok GraphQL koristi *placeOrder* mutation i zatim *orderById* query. Ovaj scenario meri kombinaciju write i read operacija i približava se realnijem toku rada aplikacije.

Tabela 3 prikazuje pregled benchmark scenarija.

Tabela 3. Benchmark scenariji

Scenario	REST operacija	GraphQL operacija	Cilj testa
Simple GET	<i>GET /products/{id}</i>	<i>productById</i> query	Osnovni per-request overhead
Deep graph fetch	<i>GET /orders/{id}</i>	<i>orderById</i> query	Dohvatanje ugnježenih podataka
Over-fetching vs minimal fetching	Pun product DTO	Samo <i>name</i> i <i>price</i>	Merenje koristi selektivnog dohvatanja
Paged product list	<i>GET /products/list?page=1&pageSize=50</i>	<i>products(page: 1, pageSize: 50)</i>	Lista od 50 proizvoda
Write + read-back	<i>POST /orders</i> + <i>GET /orders/{id}</i>	<i>placeOrder</i> + <i>orderById</i>	Kombinovana write/read operacija

Ovi scenariji zajedno pokrivaju najvažnije teorijske razlike između REST i GraphQL pristupa. Simple GET i paged list testiraju fiksne response shape-ove. Deep graph testira rad sa povezanim podacima. Minimal fetch testira GraphQL fleksibilnost. Write-read testira ponašanje kada API mora da izvrši i promenu stanja i naknadno čitanje.

6.7. Ograničenja metodologije

Iako je metodologija osmišljena tako da poređenje bude što fer, benchmark rezultati imaju određena ograničenja. Prvo ograničenje je lokalno okruženje. Svi delovi sistema izvršavaju se na istoj mašini i komuniciraju preko localhost-a. To znači da nema realne mrežne latencije, gubitka paketa, TLS terminacije, reverse proxy sloja ili cloud

infrastrukture. U produkcionom okruženju apsolutne vrednosti latencije verovatno bi bile drugačije.

Drugo ograničenje je odsustvo keširanja. REST se prirodno uklapa sa HTTP keširanjem, dok GraphQL može koristiti *persisted queries*, *client-side caching* ili specijalizovane cache strategije. Pošto nijedan od ovih mehanizama nije uključen, rezultati mere neposredno izvršavanje zahteva, a ne ponašanje sistema sa kešom. Ovo je važno jer bi keširanje moglo značajno promeniti rezultate u read-heavy scenarijima.

Treće ograničenje je odsustvo kompresije odgovora. U realnim web sistemima često se koristi gzip ili Brotli kompresija, naročito za veće JSON odgovore. Kompresija bi mogla smanjiti razlike u payload veličini, posebno kod GraphQL odgovora koji imaju dodatni envelope i ugnježdenu strukturu. U ovom radu kompresija nije korišćena kako bi se merila sirova veličina odgovora.

Četvrto ograničenje je veličina dataseta. Dataset je veći od trivijalnog primera i sadrži 1.000 proizvoda, 5.000 porudžbina i oko 17.500 stavki porudžbina, ali i dalje nije produkcionog obima. Efekti koji se pojavljuju kod miliona zapisa, kompleksnih indeksa, duboke paginacije i velikog broja istovremenih korisnika nisu u potpunosti obuhvaćeni. Ipak, dataset je dovoljno velik da osnovni scenariji budu smisleniji nego nad malom demonstracionom bazom.

Peto ograničenje odnosi se na izvršavanje REST i GraphQL testova u odvojenim pass-ovima. Ovaj pristup sprečava direktno takmičenje za resurse, ali znači da redosled izvršavanja može uticati na stanje cache-a baze podataka i operativnog sistema. U idealnom slučaju, benchmark bi se mogao ponoviti više puta uz rotaciju redosleda i prikaz prosečnih vrednosti. U okviru ovog rada rezultati se tumače uz svest o tom ograničenju.

Šesto ograničenje jeste da su REST i GraphQL implementacije konkretne implementacije u .NET okruženju. Rezultati ne govore o svim mogućim REST i GraphQL implementacijama, već o ASP.NET Core Minimal APIs i HotChocolate implementaciji nad istim domenom. Drugačiji framework, drugačiji ORM, ručno pisani SQL upiti, persisted GraphQL queries ili drugačiji DataLoader dizajn mogli bi promeniti rezultate.

Zbog ovih ograničenja rezultate treba tumačiti kao empirijski nalaz za konkretan sistem, a ne kao univerzalnu presudu o REST-u i GraphQL-u. Njihova vrednost je u tome što pokazuju kako se dva pristupa ponašaju u istom kontrolisanom .NET okruženju, nad istim domenom i pod istim benchmark pravilima.

7. Rezultati i analiza benchmark testiranja

Nakon definisanja metodologije, testnog okruženja i benchmark scenarija, izvršeno je merenje performansi REST i GraphQL API implementacija. Rezultati su dobijeni korišćenjem NBomber alata, pri čemu su za svaki scenario merene vrednosti latencije, percentila, maksimalnog vremena odgovora, veličine odgovora i broja uspešnih zahteva. Svi scenariji završeni su bez grešaka, što znači da su oba API pristupa uspešno obradila definisano opterećenje.

Analiza rezultata u ovom poglavlju organizovana je po scenarijima. Za svaki scenario najpre se prikazuje tabela sa izmerenim vrednostima, a zatim se daju tumačenje

rezultata i njihovo povezivanje sa teorijskim očekivanjima. Posebna pažnja posvećena je razlikama između prosečne latencije, p50, p95 i p99 percentila, jer ove vrednosti zajedno daju bolju sliku ponašanja sistema od samog proseka.

Važno je naglasiti da rezultati ne predstavljaju univerzalnu tvrdnju o tome da je REST ili GraphQL uvek brži. Oni predstavljaju merljive rezultate za konkretan sistem, konkretan dataset, konkretne biblioteke i konkretno lokalno okruženje. Njihova vrednost je u tome što su oba pristupa implementirana nad istim poslovnim domenom, istim podacima i istim application slojem, pa se razlike mogu smislenije analizirati.

7.1. Pregled rezultata

Ukupan pregled rezultata pokazuje da REST ostvaruje bolju latenciju u većini scenarija u kojima oba API-ja vraćaju isti ili ekvivalentan skup podataka. GraphQL ostvaruje jasnu prednost u scenariju u kome klijent traži samo minimalan skup polja, što odgovara jednoj od njegovih glavnih teorijskih prednosti. U write-read scenariju rezultati su približniji: REST je bolji na tipičnom zahtevu i p95 percentilu, dok GraphQL ima bolji p99 i maksimalnu vrednost.

Tabela 4. Zbirni pregled benchmark rezultata

#	Scenario	Brži (latencija)	Manji (payload)	Napomena
1	Simple GET	REST	REST	Oba API-ja vraćaju ista polja; GraphQL plaća dodatni parse/validate overhead
2	Deep graph fetch	REST	REST	REST koristi namenski endpoint i unapred definisan oblik odgovora
3	Over-fetch vs minimal	GraphQL	GraphQL	GraphQL traži samo dva potrebna polja
4	Paged product list	REST	REST	Oba API-ja vraćaju 50 proizvoda i isti logički response shape
5	Write + read-back	REST na p50/p95, GraphQL bolji u ekstremnom tail-u	REST	Trošak upisa u bazu dominira nad razlikom API paradigme

Iz zbirnog pregleda može se videti da rezultati uglavnom potvrđuju teorijska očekivanja. REST je efikasniji kada je oblik odgovora unapred poznat i kada server može direktno da projektuje podatke u stabilan DTO model. GraphQL pokazuje prednost u scenariju gde klijent koristi mogućnost izbora samo potrebnih polja. Time se potvrđuje da GraphQL ne treba posmatrati prvenstveno kao brži API pristup, već kao fleksibilniji pristup koji može biti efikasniji kada se njegova fleksibilnost zaista koristi.

Takođe se može primetiti da razlike u payload veličini prate očekivani obrazac. Kada oba API-ja vraćaju isti skup podataka, REST obično ima manji payload zbog jednostavnijeg JSON oblika i odsustva GraphQL data envelope-a. Kada GraphQL traži samo podskup polja, njegov payload postaje manji od REST odgovora. Zbog toga analiza veličine odgovora ima važnu ulogu u tumačenju rezultata, naročito u scenarijima gde mrežni protok može biti ograničavajući faktor.

7.2. Scenario 1: Simple GET

Prvi scenario predstavlja najjednostavniji slučaj: dohvaćanje jednog proizvoda po identifikatoru. Cilj ovog scenarija je merenje osnovnog troška API zahteva, jer se ne radi o složenom upitu, velikom odgovoru ili operaciji upisa. Oba API-ja vraćaju isti skup podataka: identifikator proizvoda, naziv, SKU, opis, cenu, identifikator kategorije i naziv kategorije.

REST varijanta koristi endpoint *GET /products/{id}*, dok GraphQL varijanta koristi *productById* query sa istim izborom polja. Pošto je skup podataka usklađen, rezultat ovog scenarija može se tumačiti kao poređenje osnovnog overhead-a REST i GraphQL pristupa.

Tabela 5. Rezultati za Scenario 1: Simple GET

Metrika	REST	GraphQL
Mean latency	4.70 ms	7.28 ms
p50	4.37 ms	7.02 ms
p95	6.69 ms	9.66 ms
p99	10.83 ms	14.18 ms
Max	40.60 ms	47.10 ms
Payload / req	0.528 KB	0.739 KB

Rezultati pokazuju da je REST brži u svim merenim vrednostima. Prosečna latencija REST zahteva iznosi 4.70 ms, dok GraphQL zahtev ima prosečnu latenciju od 7.28 ms. Razlika nije velika u apsolutnom smislu, ali je konzistentna kroz sve percentile. REST ima bolji p50, p95, p99 i maksimalno vreme odgovora.

Ovakav rezultat je očekivan. U jednostavnom scenariju, gde oba API-ja vraćaju isti skup podataka, GraphQL ne koristi svoju glavnu prednost: selektivno dohvaćanje različitih polja, već izvršava dodatni runtime pipeline. GraphQL server mora da parsira query, validira ga u odnosu na šemu, pripremi izvršavanje i oblikuje odgovor kroz standardni GraphQL response format. REST endpoint, sa druge strane, direktno mapira HTTP zahtev na handler koji vraća unapred definisan DTO.

Razlika u payload-u takođe ide u korist REST-a. REST odgovor ima veličinu 0.528 KB, dok GraphQL odgovor ima 0.739 KB. Pošto oba API-ja vraćaju ista logička polja, dodatna veličina GraphQL odgovora uglavnom potiče od standardnog *data envelope*-a i nešto drugačije JSON strukture. Kod malih odgovora, kao što je jedan proizvod, ovaj dodatni overhead procentualno izgleda veći nego kod velikih odgovora.

Ovaj scenario potvrđuje teorijsko očekivanje da REST ima prednost u jednostavnim fixed-shape zahtevima. Kada je oblik odgovora unapred poznat, REST može biti direktniji i efikasniji. GraphQL ovde ne donosi prednost, jer fleksibilnost selection set-a nije iskorišćena za smanjenje obima podataka. Zbog toga je ovaj scenario dobar prikaz osnovnog troška GraphQL izvršnog modela u odnosu na jednostavan REST endpoint.

Ipak, treba naglasiti da razlika od nekoliko milisekundi u lokalnom okruženju ne mora uvek biti presudna u realnom sistemu. U produkcionom okruženju mrežna latencija, TLS, gateway sloj, autentifikacija i drugi faktori mogu relativno smanjiti značaj razlike između 4.70 ms i 7.28 ms. Međutim, za akademsko poređenje i razumevanje osnovnog overhead-a, rezultat je jasan: REST je u ovom scenariju efikasniji.

7.3. Scenario 2: Deep graph fetch

Drugi scenario meri dohvaćanje složenijeg objektnog grafa. U ovom slučaju klijent traži porudžbinu sa povezanim podacima, uključujući osnovne informacije o porudžbini, kupcu i stavkama porudžbine sa proizvodima. Ovakav scenario je važan jer se GraphQL često navodi kao posebno pogodan za rad sa ugnježđenim podacima. Klijent može u jednom upitu da navede povezanu strukturu koju želi da dobije.

Da bi poređenje bilo fer, REST implementacija takođe ima namenski endpoint koji vraća isti logički oblik podataka u jednom HTTP pozivu. Time se ne poredi GraphQL protiv naivnog REST pristupa koji bi zahtevao više round-trip poziva, već GraphQL protiv dobro projektovanog REST endpoint-a prilagođenog konkretnom prikazu. Ovo je važna metodološka odluka, jer cilj nije dokazati da je jedan pristup bolji na osnovu loše implementacije drugog.

Tabela 6. Rezultati za Scenario 2: Deep graph fetch

Metrika	REST	GraphQL
Mean latency	4.98 ms	8.07 ms
p50	4.68 ms	6.98 ms
p95	6.66 ms	10.74 ms
p99	10.74 ms	30.45 ms
Max	102.58 ms	120.58 ms
Payload / req	1.078 KB	1.348 KB

Rezultati pokazuju da REST i u ovom scenariju ima bolje vrednosti kroz sve glavne metrike. Prosečna latencija REST zahteva iznosi 4.98 ms, dok GraphQL zahtev ima prosečnu latenciju od 8.07 ms. P50 je takođe bolji kod REST-a: 4.68 ms naspram 6.98 ms. Razlika postaje izraženija kod p99 vrednosti, gde REST ima 10.74 ms, a GraphQL 30.45 ms.

Ovakav rezultat pokazuje da GraphQL ne pobeđuje automatski u svakom scenariju sa ugnježđenim podacima. Njegova prednost je u tome što klijent može fleksibilno da zatraži povezanu strukturu, ali ako REST već ima namenski endpoint koji vraća isti oblik

podataka u jednom pozivu, REST može biti efikasniji. Razlog je u tome što REST može unapred pripremiti query i DTO projekciju za tačno taj scenario, dok GraphQL mora da prođe kroz resolver i execution pipeline.

REST u ovom slučaju koristi unapred poznat oblik odgovora. Server zna da treba da vrati porudžbinu, kupca i stavke porudžbine sa osnovnim informacijama o proizvodu. Zato može koristiti unapred definisan EF Core query sa odgovarajućim *Include* ili projekcionim pristupom. GraphQL, sa druge strane, mora da interpretira selection set i izvrši tražena polja kroz GraphQL runtime. Iako je rezultat logički isti, put do rezultata nije isti.

Payload je takođe manji kod REST-a. REST odgovor ima 1.078 KB, dok GraphQL odgovor ima 1.348 KB. Razlika od oko 25% potiče iz GraphQL envelope-a i načina strukturiranja odgovora. Kod ugnježenih objekata dodatna struktura može postati vidljivija nego kod ravnog DTO modela.

Ovaj scenario je posebno važan za tumačenje popularne tvrdnje da je GraphQL bolji za dohvaćanje povezanih podataka. Ta tvrdnja je tačna u situaciji kada REST klijent mora da šalje više zahteva da bi sastavio isti prikaz. Međutim, ako REST API ima namenski endpoint koji u jednom zahtevu vraća potrebnu strukturu, prednost GraphQL-a u broju round-trip poziva nestaje, a dodatni execution overhead postaje vidljiviji.

Zaključak za ovaj scenario je da GraphQL pruža veću fleksibilnost za definisanje oblika ugnježenog odgovora, ali REST može biti brži kada je konkretan oblik odgovora unapred poznat i optimizovan. Ovo je jedan od centralnih nalaza rada: prednosti GraphQL-a zavise od toga da li klijent zaista koristi njegovu fleksibilnost, dok REST ostaje veoma efikasan za namenski projektovane fixed-shape endpoint-e.

7.4. Scenario 3: Over-fetching vs minimal fetching

Treći scenario je osmišljen tako da prikaže jednu od najvažnijih teorijskih prednosti GraphQL-a: mogućnost da klijent zatraži samo ona polja koja su mu zaista potrebna. U ovom scenariju klijentu treba samo naziv i cena proizvoda. REST varijanta koristi postojeći endpoint koji vraća puni product DTO, dok GraphQL varijanta šalje query koji traži samo dva polja: *name* i *price*.

Za razliku od prethodna dva scenarija, ovaj scenario namerno nije simetričan u pogledu oblika odgovora (response shape-a). Njegova svrha nije da proveri koji pristup je brži kada vraćaju isti odgovor, već da pokaže šta se dešava kada GraphQL koristi svoju osnovnu prednost. U realnim aplikacijama često postoje situacije u kojima klijentu treba samo mali deo podataka, a REST endpoint vraća širi objekat jer je tako unapred definisan.

Tabela 7. Rezultati za Scenario 3: Over-fetching vs minimal fetching

Metrika	REST (pun DTO)	GraphQL (2 polja)
Mean latency	4.53 ms	4.09 ms
p50	4.02 ms	3.94 ms

p95	6.28 ms	5.57 ms
p99	14.20 ms	7.34 ms
Payload / req	0.528 KB	0.460 KB

Rezultati pokazuju da GraphQL u ovom scenariju ostvaruje bolji rezultat u svim prikazanim metrikama. Prosečna latencija GraphQL zahteva iznosi 4.09 ms, dok REST ima 4.53 ms. Razlika u prosečnoj latenciji nije velika, ali je zanimljivo da GraphQL pobeđuje i na p50, p95 i p99 vrednostima. Posebno je značajna p99 vrednost: GraphQL ima 7.34 ms, dok REST ima 14.20 ms.

Payload je takođe manji kod GraphQL-a. GraphQL odgovor ima 0.460 KB, dok REST odgovor ima 0.528 KB. Iako GraphQL ima dodatni *data* envelope, u ovom slučaju to nije dovoljno da poništi prednost dobijenu slanjem manjeg broja polja. Pošto REST vraća puni product DTO, on prenosi i podatke koji klijentu nisu potrebni.

Ovaj scenario potvrđuje da GraphQL selektivnost može imati merljiv efekat. Kada klijent traži samo mali podskup polja, GraphQL ne samo da smanjuje veličinu odgovora, već može smanjiti i vreme obrade. To zavisi od toga da li server zaista koristi projekcije i ne materijalizuje nepotrebna polja. U implementaciji CommerceHub sistema GraphQL je podešen tako da selection set može uticati na izvršavanje upita, što omogućava da se ova prednost vidi u rezultatu.

Sa stanovišta REST-a, ovaj scenario prikazuje tipičan over-fetching problem. REST endpoint vraća unapred definisan oblik proizvoda, bez obzira na to što klijentu trebaju samo naziv i cena. Naravno, REST bi mogao rešiti ovaj problem uvođenjem posebnog endpoint-a, na primer *GET /products/{id}/summary*, ili uvođenjem mehanizma za izbor polja. Međutim, to bi zahtevalo dodatni API dizajn i potencijalno povećalo broj endpoint-a ili složenost REST implementacije.

Zbog toga se ovaj scenario može tumačiti kao najjači praktični argument u korist GraphQL-a u okviru rada. On pokazuje da GraphQL nije samo drugačiji način pisanja API zahteva, već može omogućiti efikasniju komunikaciju kada se koristi za ono za šta je posebno pogodan: precizno definisanje potrebnih polja od strane klijenta.

Ipak, ovaj rezultat ne treba preuveličavati. GraphQL pobeđuje u scenariju koji je za njega posebno povoljan. To ne znači da će biti bolji u svim scenarijima, već da njegova prednost zavisi od toga da li aplikacija zaista ima potrebe za selektivnim dohvatanjem podataka. Ako svi klijenti uvek traže pun oblik objekta, ova prednost se smanjuje ili nestaje.

7.5. Scenario 4: Paged product list

Četvrti scenario predstavlja tipičan ekran liste proizvoda. Oba API-ja vraćaju prvu stranu proizvoda, sa parametrima *page = 1* i *pageSize = 50*. Response shape je usklađen tako da oba pristupa vraćaju isti logički skup polja: identifikator proizvoda, naziv, SKU, opis, cenu i naziv kategorije. Ovaj scenario je važan jer meri ponašanje API-ja kada se vraća veći broj objekata, ali bez složenog ugnježđenog grafa.

Tabela 8. Rezultati za Scenario 4: Paged product list

Metrika	REST	GraphQL
Mean latency	5.42 ms	9.70 ms
p50	5.27 ms	8.02 ms
p95	7.52 ms	12.23 ms
p99	8.86 ms	67.58 ms
Max	14.42 ms	111.20 ms
Payload / req	13.553 KB	14.016 KB

Rezultati pokazuju da REST ostvaruje bolje performanse u ovom scenariju. Prosečna latencija REST zahteva iznosi 5.42 ms, dok GraphQL ima 9.70 ms. P50 je takođe bolji kod REST-a: 5.27 ms naspram 8.02 ms. Razlika je posebno vidljiva kod p99 vrednosti, gde REST ima 8.86 ms, dok GraphQL ima 67.58 ms.

Payload je veoma sličan, ali REST je i tu nešto bolji. REST odgovor ima 13.553 KB, dok GraphQL odgovor ima 14.016 KB. Razlika je relativno mala, oko tri procenta, što pokazuje da je response shape dobro usklađen. Upravo zato se razlika u latenciji može tumačiti kao razlika u načinu obrade, a ne kao posledica značajno različite količine podataka.

REST prednost u ovom scenariju može se objasniti jednostavnijim modelom izvršenja. REST endpoint izvršava direktan upit koji sortira, preskače i uzima 50 proizvoda, zatim ih projektuje u *Data Transfer Objekat* i vraća JSON listu. GraphQL mora da obradi upit, validira selection set, primeni projekciju i oblikuje odgovor kroz GraphQL response format. Iako HotChocolate projekcije mogu optimizovati čitanje podataka, tok izvršenja i dalje ima dodatne korake.

P99 vrednost kod GraphQL-a pokazuje da postoji izraženiji tail latency. Većina zahteva je i dalje relativno brza, ali najsporijih jedan procenat zahteva značajno odstupa. To može biti posledica kombinacije GraphQL izvršnog pipeline-a, resolver/projection logike, serijalizacije većeg broja objekata i lokalnih runtime efekata. REST u ovom scenariju pokazuje stabilniji tail.

Ovaj rezultat potvrđuje da REST ima prednost u scenarijima gde se vraća unapred poznata lista podataka sa fiksnim oblikom. Ako server zna da treba da vrati 50 proizvoda sa istim skupom polja, REST endpoint može biti vrlo jednostavan i efikasan. GraphQL zadržava prednost fleksibilnosti, jer klijent može promeniti selection set, ali u ovom konkretnom fixed-shape scenariju ta fleksibilnost nije iskorišćena za smanjenje obima podataka.

Scenario 4 je zato značajan za praktične smernice. Ako aplikacija ima standardne liste sa stabilnim kolonama, REST može biti jednostavniji i brži izbor. Ako različiti klijenti često traže različite kolone liste, GraphQL može biti opravdan zbog fleksibilnosti, čak i ako pojedinačni fiksni upit ima nešto veći overhead.

7.6. Scenario 5: Write + read-back

Peti scenario predstavlja kombinovanu operaciju upisa i čitanja. U svakoj iteraciji najpre se kreira nova porudžbina, a zatim se kreirana porudžbina dohvata nazad. REST varijanta koristi dva HTTP poziva: *POST /orders* za kreiranje porudžbine i *GET /orders/{id}* za dohvaćanje. GraphQL varijanta koristi *placeOrder* mutation koja vraća identifikator porudžbine, a zatim *orderById* query sa selection set-om usklađenim sa REST read-back odgovorom.

Ovaj scenario se razlikuje od prethodnih jer uključuje upis u bazu podataka. Operacije upisa su skuplje od jednostavnog čitanja jer uključuju validaciju, kreiranje entiteta, upis u bazu i čuvanje promena. Zbog toga se ovaj scenario izvršava sa manjim opterećenjem od read scenarija.

Tabela 9. Rezultati za Scenario 5: Write + read-back

Metrika	REST	GraphQL
Mean latency	24.14 ms	28.55 ms
p50	15.82 ms	24.05 ms
p95	45.57 ms	49.41 ms
p99	166.27 ms	137.22 ms
Max	870.01 ms	386.84 ms
Payload / req	0.889 KB	1.160 KB

Rezultati pokazuju da REST ima bolju prosečnu latenciju, bolji p50 i bolji p95. Prosečna latencija REST varijante iznosi 24.14 ms, dok GraphQL ima 28.55 ms. Na tipičnom zahtevu razlika je izraženija: REST p50 iznosi 15.82 ms, dok GraphQL ima 24.05 ms. To pokazuje da je REST brži u uobičajenom toku izvršavanja ovog scenarija.

Međutim, GraphQL ima bolji p99 i bolju maksimalnu vrednost. P99 kod GraphQL-a iznosi 137.22 ms, dok REST ima 166.27 ms. Maksimalna vrednost kod GraphQL-a je 386.84 ms, dok REST ima 870.01 ms. To znači da REST ima bolje ponašanje u većini zahteva, ali GraphQL u ovom testu ima stabilniji ekstremni tail. Ovakav rezultat pokazuje zašto je važno posmatrati više metrika, a ne samo prosečnu latenciju.

Dominantan trošak u ovom scenariju je upis u bazu podataka. Za razliku od simple GET ili product list scenarija, ovde API overhead nije jedini značajan faktor. Kreiranje porudžbine podrazumeva formiranje *order* entiteta, stavki porudžbine, računanje ukupnog iznosa i poziv *SaveChanges*. Nakon toga sledi dodatni read-back poziv. Kada baza postane dominantan deo ukupnog vremena, razlika između REST i GraphQL protokola postaje manje izražena.

Payload je manji kod REST-a. REST odgovor ima 0.889 KB, dok GraphQL ima 1.160 KB. Razlika je, opet, posledica GraphQL response envelope-a i strukture odgovora. Pošto oba pristupa rade write i read-back, GraphQL ne koristi značajnu selektivnu prednost u ovom scenariju, već vraća oblik odgovora usklađen sa REST-om.

Ovaj scenario pokazuje da se rezultati mogu razlikovati zavisno od prirode operacije. Kod čistih read scenarija API execution overhead može biti dominantniji. Kod

write scenarija veći deo vremena odlazi na bazu i čuvanje promena. Zbog toga se REST i GraphQL ovde ponašaju približnije nego što bi se možda očekivalo na osnovu jednostavnih read scenarija.

Zaključak za ovaj scenario jeste da je REST bolji na tipičnom zahtevu i p95 vrednosti, dok GraphQL pokazuje stabilniji ekstremni tail. U praktičnom smislu, oba pristupa su u ovom scenariju upotrebljiva, a izbor između njih više bi zavisio od šireg API dizajna i potreba klijenta nego od same razlike u performansama.

7.7. Analiza rezultata i poređenje sa teorijskim očekivanjima

Nakon pojedinačne analize svih benchmark scenarija, moguće je izvesti opštije zaključke o ponašanju REST i GraphQL pristupa u implementiranom sistemu. Posebno su značajne dve grupe metrika: veličina odgovora i latencija. Veličina odgovora pokazuje koliko podataka se prenosi između servera i klijenta, dok latencija pokazuje koliko vremena je potrebno da server obradi zahtev i vrati odgovor. Tek zajedničkim posmatranjem ovih metrika može se dobiti potpunija slika o praktičnim razlikama između dva pristupa.

Kada se posmatra veličina odgovora, REST uglavnom ima prednost u scenarijima u kojima oba API-ja vraćaju isti logički skup podataka. U prvom scenariju, gde se dohvaća jedan proizvod, REST odgovor ima veličinu 0.528 KB, dok GraphQL odgovor ima 0.739 KB. Sličan odnos se vidi i u drugom scenariju, gde REST payload iznosi 1.078 KB, a GraphQL 1.348 KB. U petom scenariju REST takođe vraća manji odgovor, 0.889 KB naspram 1.160 KB kod GraphQL-a. Ovi rezultati su očekivani, jer GraphQL odgovor sadrži standardni *data envelope* i često nešto dublju JSON strukturu, dok REST najčešće vraća direktniji DTO oblik.

Kod većih odgovora, međutim, relativna razlika u veličini odgovora postaje manja. U četvrtom scenariju, gde oba API-ja vraćaju listu od 50 proizvoda, REST payload iznosi 13.553 KB, dok GraphQL payload iznosi 14.016 KB. Razlika postoji, ali je mala, što pokazuje da je response shape dobro usklađen i da fiksni GraphQL overhead postaje manje značajan kada se u odgovoru nalazi veći broj objekata. To znači da se GraphQL envelope najviše primećuje kod malih odgovora, dok kod većih odgovora njegov relativni udeo opada.

Najvažniji izuzetak je treći scenario, odnosno over-fetching vs minimal fetching. U tom slučaju GraphQL ima manji payload od REST-a: 0.460 KB naspram 0.528 KB. Razlog je u tome što GraphQL traži samo dva polja, *name* i *price*, dok REST vraća pun product DTO. Iako GraphQL i dalje ima *data envelope*, ukupna količina prenetih podataka je manja jer se ne vraćaju nepotrebna polja. Ovaj rezultat potvrđuje jednu od glavnih teorijskih prednosti GraphQL-a: kada klijent zaista koristi mogućnost selektivnog dohvaćanja podataka, GraphQL može smanjiti veličinu odgovora.

Sličan obrazac se vidi i kod latencije. U scenarijima sa fiksnim i unapred poznatim oblikom odgovora REST uglavnom ostvaruje bolje rezultate. U simple GET scenariju REST ima prosečnu latenciju od 4.70 ms, dok GraphQL ima 7.28 ms. P50 vrednost kod REST-a iznosi 4.37 ms, a kod GraphQL-a 7.02 ms. U deep graph scenariju REST takođe ima bolji

mean, p50, p95 i p99, dok u paged product list scenariju REST pokazuje posebno stabilan tail latency: p99 iznosi 8.86 ms, dok GraphQL p99 iznosi 67.58 ms. Ovi rezultati ukazuju na to da REST ima manji i stabilniji overhead kada server unapred zna koji oblik odgovora treba da vrati.

Razlog za ovakvo ponašanje nalazi se u različitom toku izvršavanja zahteva. REST zahtev prolazi kroz HTTP routing, poziva odgovarajući endpoint, izvršava application handler i vraća DTO response. GraphQL zahtev, čak i kada vraća isti skup podataka, mora da prođe kroz dodatne faze: parsiranje upita, validaciju prema šemi, planiranje izvršavanja, resolver ili projection pipeline i oblikovanje odgovora prema GraphQL specifikaciji. Ove faze omogućavaju fleksibilnost, ali uvode dodatni trošak koji se vidi u fixed-shape scenarijima.

Međutim, treći scenario pokazuje da GraphQL overhead može biti nadoknađen kada selection set zaista smanjuje obim posla. U over-fetching vs minimal fetching scenariju GraphQL ima bolji mean, p50, p95 i p99. P50 vrednost iznosi 3.94 ms kod GraphQL-a naspram 4.02 ms kod REST-a, dok je p99 kod GraphQL-a 7.34 ms, a kod REST-a 14.20 ms. Ovo pokazuje da GraphQL može biti ne samo fleksibilniji, već i brži kada klijent traži samo mali podskup dostupnih polja. Drugim rečima, GraphQL nije automatski sporiji; njegova efikasnost zavisi od toga da li se njegova glavna prednost zaista koristi.

Write-read scenario donosi mešoviti rezultat. REST ima bolji mean, p50 i p95, dok GraphQL ima bolji p99 i maksimalnu vrednost. To ukazuje na to da REST bolje radi u tipičnom toku izvršavanja, ali je u ovom konkretnom testu imao veće ekstremne skokove. Pošto ovaj scenario uključuje upis u bazu, njegovo tumačenje je složenije nego kod čistih read scenarija. Značajan deo ukupnog vremena odlazi na kreiranje entiteta, rad Entity Framework Core-a i *SaveChanges* operaciju, pa razlika između API pristupa postaje samo jedan deo ukupnog troška.

Kada se rezultati uporede sa teorijskim očekivanjima, može se zaključiti da ih benchmark uglavnom potvrđuje. Teorijski se očekuje da REST bude efikasniji u jednostavnim i unapred definisanim scenarijima, jer nema dodatni sloj za parsiranje i izvršavanje klijentski definisanih upita. To je potvrđeno u simple GET, deep graph i paged product list scenarijima. Takođe se očekuje da GraphQL ima prednost kada klijent želi da izbegne over-fetching i zatraži samo potrebna polja. To je potvrđeno u trećem scenariju, gde GraphQL vraća manji payload i ostvaruje bolju latenciju.

Posebno je važno tumačenje deep graph scenarija. GraphQL se često opisuje kao pristup koji je pogodan za dohvatanje povezanih i ugnjeđenih podataka. To jeste tačno u situaciji kada bi REST klijent morao da šalje više zahteva kako bi sastavio isti prikaz. Međutim, u ovom radu REST nije implementiran naivno, već kroz namenski endpoint koji vraća isti objektni graf u jednom zahtevu. Kada se porede dobro projektovan REST endpoint i GraphQL upit koji vraćaju isti skup podataka, REST može biti brži zbog jednostavnijeg modela izvršavanja. Time se precizira teorijska tvrdnja: GraphQL ima prednost kod ugnjeđenih podataka prvenstveno kada njegova fleksibilnost smanjuje broj poziva ili omogućava različite oblike odgovora bez dodatnih endpoint-a.

Na osnovu celokupne analize može se zaključiti da REST i GraphQL ne treba posmatrati kao univerzalno bolji ili lošiji pristup. REST se u ovom radu pokazao efikasnijim u scenarijima sa stabilnim i unapred poznatim response modelima. GraphQL se pokazao boljim u scenariju u kome klijent traži samo mali broj polja, čime se smanjuju

i veličina odgovora i vreme obrade. Izbor između ova dva pristupa zato treba vezati za realne obrasce korišćenja API-ja. Ako sistem ima stabilne klijente i fiksne oblike odgovora, REST je često jednostavniji i efikasniji. Ako sistem ima više različitih klijenata koji često traže različite projekcije istih podataka, GraphQL može pružiti veću fleksibilnost i smanjiti over-fetching.

8. Diskusija i zaključak

8.1. Diskusija rezultata

Rezultati sprovedenog benchmark testiranja pokazuju da REST i GraphQL imaju različite performansne profile i da njihova pogodnost zavisi od konkretnog scenarija. REST je u implementiranom sistemu CommerceHub pokazao prednost u većini scenarija sa unapred definisanim oblikom odgovora. GraphQL je pokazao jasnu prednost u scenariju gde klijent traži samo minimalan skup polja. Ovakav rezultat je u skladu sa osnovnim karakteristikama oba pristupa.

REST je jednostavniji i direktniji. Kada endpoint ima stabilan response model, server može da izvrši optimizovan query, projektuje rezultat u DTO i vrati jednostavan JSON odgovor. U takvim okolnostima nema dodatnog sloja parsiranja i validacije upita. Zato REST često ima nižu latenciju i manji payload kada se vraća isti skup podataka.

GraphQL je fleksibilniji. Njegova glavna prednost nije nužno sirova brzina, već mogućnost da klijent sam odredi oblik odgovora. Kada se ta mogućnost koristi, kao u over-fetching scenariju, GraphQL može biti efikasniji i po latenciji i po veličini odgovora. Međutim, kada se GraphQL koristi za vraćanje istog fixed-shape odgovora kao REST, njegov dodatni execution pipeline može dovesti do veće latencije.

Ovi rezultati pokazuju da se REST i GraphQL ne mogu kvalitetno porediti samo kroz opšte tvrdnje. Tvrdnja da je REST brži može biti tačna u fiksnim scenarijima, ali ne mora važiti kada GraphQL vraća znatno manje podataka. Tvrdnja da je GraphQL bolji za ugnježdene podatke može biti tačna ako REST zahteva više poziva, ali ne mora važiti ako REST ima namenski endpoint. Zbog toga je potrebno uvek posmatrati konkretan domen, konkretne klijentske potrebe i konkretan oblik zahteva.

Praktični deo rada takođe pokazuje da implementacioni detalji značajno utiču na rezultate. U toku usklađivanja scenarija pokazalo se da različit broj vraćenih proizvoda ili različit response shape mogu potpuno promeniti tumačenje benchmark-a. Zbog toga je fer poređenje zahtevalo više iteracija i pažljivo poravnanje REST i GraphQL zahteva. Ovaj proces je važan rezultat sam po sebi, jer pokazuje koliko je lako dobiti pogrešnu sliku ako se API pristupi porede površno.

Sa akademskog stanovišta, najvažniji doprinos rada je u tome što kombinuje teorijsku analizu sa praktičnom implementacijom i eksperimentalnim merenjem. Teorija objašnjava zašto se očekuju određene razlike, implementacija omogućava da se te razlike testiraju, a benchmark rezultati pokazuju kako se one zaista manifestuju u konkretnom .NET okruženju.

8.2. Praktične smernice za izbor REST ili GraphQL pristupa

Na osnovu teorijske analize i benchmark rezultata mogu se formulisati praktične smernice za izbor između REST i GraphQL pristupa. Ove smernice ne treba posmatrati kao stroga pravila, već kao pomoć pri donošenju arhitektonske odluke.

REST je pogodan izbor kada sistem ima stabilne i predvidive API operacije. Ako klijenti uglavnom traže iste oblike podataka, REST omogućava jednostavne i efikasne endpoint-e. Na primer, ako aplikacija ima jasno definisane ekrane za listu proizvoda, detalje proizvoda, listu porudžbina i detalje porudžbine, REST endpoint-i mogu biti direktno prilagođeni tim potrebama. U takvim okolnostima REST pruža dobru kombinaciju performansi, jednostavnosti i održivosti.

REST je takođe pogodan kada je važna jednostavna integracija sa postojećom HTTP infrastrukturom. Alati za dokumentovanje, testiranje, keširanje, monitoring i gateway konfiguraciju često prirodno podržavaju REST model. HTTP metode i status kodovi daju jasnu semantiku, a endpoint-i su lako razumljivi i ljudima i alatima.

REST može biti bolji izbor i kada tim želi strogo kontrolisane API odgovore. Pošto server definiše svaki response model, lakše je predvideti cenu izvršavanja endpoint-a, veličinu odgovora i pravila autorizacije. Ovo je važno u sistemima gde je potrebno precizno upravljanje performansama i sigurnošću.

GraphQL je pogodniji kada postoji više različitih klijenata sa različitim potrebama za podacima. Na primer, web aplikacija, mobilna aplikacija i partnerski sistem mogu koristiti isti backend, ali tražiti različite skupove polja. Umesto da se za svaki klijent uvode posebni REST endpoint-i ili proširuju postojeći DTO modeli, GraphQL omogućava svakom klijentu da definiše selection set prema svojim potrebama.

GraphQL je posebno koristan kada je over-fetching izražen problem. Ako REST endpoint-i često vraćaju više podataka nego što je klijentu potrebno, GraphQL može smanjiti payload i potencijalno poboljšati performanse. Benchmark rezultati ovog rada potvrđuju da u scenariju minimalnog dohvatanja podataka GraphQL može biti bolji i po latenciji i po veličini odgovora.

GraphQL može biti pogodan i kada se često menjaju korisnički interfejsi. Ako frontend timovi redovno traže nove kombinacije polja, GraphQL šema može omogućiti brže iteracije bez stalnog dodavanja novih endpoint-a. Međutim, ovo važi samo ako je šema dobro projektovana i ako backend implementacija ima odgovarajuće mehanizme za projekcije, DataLoader-e, paginaciju i kontrolu kompleksnosti upita.

U praksi, izbor ne mora uvek biti isključiv. Neki sistemi mogu koristiti REST za stabilne operacije i GraphQL za složenije klijentske prikaze. Ipak, kombinovanje oba pristupa uvodi dodatnu složenost i treba ga opravdati konkretnim potrebama. U okviru ovog rada oba pristupa postoje paralelno radi poređenja, ali u realnom sistemu odluka bi zavisila od timske organizacije, potreba klijenata i dugoročnog plana razvoja API-ja.

8.3. Zaključak

U ovom radu izvršena je komparativna analiza REST i GraphQL pristupa u razvoju Web API sistema u .NET okruženju. Analiza je obuhvatila teorijske osnove oba pristupa, implementaciju praktičnog sistema CommerceHub i eksperimentalno poređenje performansi korišćenjem NBomber alata. REST API implementiran je pomoću ASP.NET Core Minimal APIs tehnologije, dok je GraphQL API implementiran pomoću HotChocolate biblioteke. Oba API-ja rade nad istim poslovnim domenom, istom bazom podataka i istim application slojem.

Teorijska analiza pokazala je da REST i GraphQL polaze od različitih principa. REST se zasniva na resursima, endpoint-ima i unapred definisanim odgovorima. GraphQL se zasniva na šemi, selection set-u i klijentski definisanom obliku odgovora. REST nudi jednostavnost, zrelost i efikasnost u stabilnim scenarijima, dok GraphQL nudi veću fleksibilnost i mogućnost preciznog dohvaćanja podataka.

Praktični deo rada pokazao je da ove teorijske razlike imaju merljive posledice. U scenarijima gde oba API-ja vraćaju isti fixed-shape odgovor, REST je uglavnom ostvario nižu latenciju i manji payload. To se vidi u simple GET, deep graph i paged product list scenarijima. Razlog je jednostavniji execution model REST-a i odsustvo dodatnog GraphQL parsing, validation i execution overhead-a.

Sa druge strane, GraphQL je ostvario jasnu prednost u over-fetching vs minimal fetching scenariju. Kada je klijentu bio potreban samo naziv i cena proizvoda, GraphQL je vratio manji odgovor i ostvario bolju latenciju. Ovaj rezultat potvrđuje jednu od glavnih prednosti GraphQL-a: mogućnost da klijent dobije tačno ona polja koja su mu potrebna. Time se pokazuje da GraphQL ima najveću vrednost kada se njegova fleksibilnost zaista koristi.

Write-read scenario pokazao je da kod operacija koje uključuju upis u bazu razlika između API pristupa može biti manje izražena. REST je bio bolji na tipičnom zahtevu i p95 percentilu, dok je GraphQL imao bolji ekstremni tail. Ovaj rezultat ukazuje na to da kod složenijih operacija ukupno vreme zavisi ne samo od API sloja, već i od baze podataka, ORM-a i samog poslovnog toka.

Na osnovu sprovedene analize može se zaključiti da REST i GraphQL ne treba posmatrati kao univerzalno bolji ili lošiji pristup. REST je pogodniji za stabilne i unapred poznate API odgovore, naročito kada su važni jednostavnost, manji overhead i dobra integracija sa HTTP infrastrukturom. GraphQL je pogodniji kada klijenti imaju različite potrebe za podacima, kada je važno smanjiti over-fetching i kada fleksibilnost šeme donosi stvarnu razvojnu prednost.

Konačni zaključak rada jeste da izbor između REST i GraphQL pristupa treba donositi na osnovu konkretnih zahteva sistema, a ne na osnovu opštih tvrdnji ili tehnoloških trendova. Ako sistem ima predvidive klijente i stabilne response modele, REST može biti jednostavniji i efikasniji izbor. Ako sistem ima više različitih klijenata, promenljive potrebe za podacima i česte zahteve za različitim projekcijama, GraphQL može pružiti značajne prednosti. U oba slučaja, kvalitet implementacije, usklađenost oblika odgovora, način pristupa podacima i jasno definisana metodologija merenja imaju presudan uticaj na konačne rezultate.

9. Literatura

- [1] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral dissertation, University of California, Irvine, 2000.
- [2] Microsoft, “APIs overview — ASP.NET Core,” Microsoft Learn.
[Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis?view=aspnetcore-10.0>
- [3] Microsoft, “Tutorial: Create a Minimal API with ASP.NET Core,” Microsoft Learn.
[Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api?view=aspnetcore-10.0&tabs=visual-studio>
- [4] Microsoft, “Overview of Entity Framework Core,” Microsoft Learn.
[Online]. Available: <https://learn.microsoft.com/en-us/ef/core>
- [5] Microsoft, “.NET Observability with OpenTelemetry,” Microsoft Learn.
[Online]. Available: <https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel>
- [6] GraphQL Foundation, “GraphQL Specification,” GraphQL official specification.
[Online]. Available: <https://spec.graphql.org/September2025>
- [7] GraphQL Foundation, “Queries,” GraphQL official documentation.
[Online]. Available: <https://graphql.org/learn/queries>
- [8] GraphQL Foundation, “Mutations,” GraphQL official documentation.
[Online]. Available: <https://graphql.org/learn/mutations>
- [9] NBomber, “Overview,” NBomber documentation.
[Online]. Available: <https://nbomber.com/docs/getting-started/overview>
- [10] ChilliCream, “Hot Chocolate v16 Documentation,” ChilliCream GraphQL Platform documentation.
[Online]. Available: <https://chillicream.com/docs/hotchocolate/v16>
- [11] OpenTelemetry, “OpenTelemetry Documentation,” official documentation.
[Online]. Available: <https://opentelemetry.io/docs>

10. Spisak slika, tabela i listinga

Slika 1. ER dijagram CommerceHub domena	32
Slika 2. Arhitektura CommerceHub sistema	33
Slika 3. Sekvenca pokretanja sistema kroz .NET Aspire.....	36
Slika 4. Tok REST zahteva	47
Slika 5. Tok GraphQL zahteva	47
Tabela 1. Komparativni pregled REST i GraphQL pristupa.....	22
Tabela 2. Dataset korišćen u praktičnom delu rada.....	35
Tabela 3. Benchmark scenariji.....	54
Tabela 4. Zbirni pregled benchmark rezultata.....	56
Tabela 5. Rezultati za Scenario 1: Simple GET	57
Tabela 6. Rezultati za Scenario 2: Deep graph fetch.....	58
Tabela 7. Rezultati za Scenario 3: Over-fetching vs minimal fetching	59
Tabela 8. Rezultati za Scenario 4: Paged product list.....	61
Tabela 9. Rezultati za Scenario 5: Write + read-back.....	62
Listing 1. Primer REST endpoint-a.....	24
Listing 2. Primer GraphQL query metode.....	25
Listing 3. Response model proizvoda	38
Listing 4. Primer query handler-a za dohvaćanje proizvoda.....	39
Listing 5. REST handler za listu proizvoda.....	40
Listing 6. REST endpoint za listu proizvoda	41
Listing 7. Primer GraphQL query metode za dohvaćanje proizvoda po identifikatoru	42
Listing 8. GraphQL upit za proizvod sa istim poljima kao REST odgovor	43
Listing 9. GraphQL upit sa minimalnim skupom polja.....	44